NLP LAB REPORT

Register No: CH.SC.R4CSE25003-PT NAME: Muni Koteswararao Chevuru
Amrita Vishwa Vidyapeetham, Chennai campus

EXPERIMENT-1
Tokenization

pip install nltk==3.5

```
Requirement already satisfied: nltk==3.5 in /usr/local/lib/python3.12/dist-packages (3.5)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk==3.5) (8.3.2)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk==3.5) (1.5.3)
Requirement already satisfied: regex in /usr/local/lib/python3.12/dist-packages (from nltk==3.5) (2025.11.3
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk==3.5) (4.67.3)
```

pip install numpy matplotlib

```
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
```

```
from nltk.tokenize import sent_tokenize, word_tokenize
import nltk
nltk.download('punkt')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
True
```

```
example_string = """Muad'Dib learned rapidly because his first training was in how to learn.
And the first lesson of all was the basic trust that he could learn.
It's shocking to find how many people do not believe they can learn,
and how many more believe learning to be difficult."""
sent_tokenize(example_string)
```

```
["Muad'Dib learned rapidly because his first training was in how to learn.",
 'And the first lesson of all was the basic trust that he could learn.',
 "It's shocking to find how many people do not believe they can learn,\nand how many more believe learning to be difficult."]
```

```
nltk.download("stopwords")
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
True
```

```
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
worf_quote = "Sir, I protest. I am not a merry man!"
words_in_quote = word_tokenize(worf_quote)
words_in_quote
```

```
['Sir', ',', 'I', 'protest', '.', 'I', 'am', 'not', 'a', 'merry', 'man', '!']
```

```
stop_words = set(stopwords.words("english"))
filtered_list = []
for word in words_in_quote:
    if word.casefold() not in stop_words:
```

```
    filtered_list.append(word)
print(filtered_list)
```

```
['Sir', ',', 'protest', '.', 'merry', 'man', '!']
```

```
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize
stemmer = PorterStemmer()
string_for_stemming = """
.. The crew of the USS Discovery discovered many discoveries.
... Discovering is what explorers do."""
words = word_tokenize(string_for_stemming)
words
```

```
['..',
 'The',
 'crew',
 'of',
 'the',
 'USS',
 'Discovery',
 'discovered',
 'many',
 'discoveries',
 '.',
 '...',
 'Discovering',
 'is',
 'what',
 'explorers',
 'do',
 '.']
```

```
stemmed_words = [stemmer.stem(word) for word in words]
stemmed_words
```

```
['..',
 'the',
 'crew',
 'of',
 'the',
 'uss',
 'discoveri',
 'discov',
 'mani',
 'discoveri',
 '.',
 '...',
 'discov',
 'is',
 'what',
 'explor',
 'do',
 '.']
```

Tagging Parts of Speech

```
from nltk.tokenize import word_tokenize
import nltk
nltk.download('averaged_perceptron_tagger') # Download the missing resource

sagan_quote = """
If you wish to make an apple pie from scratch,
... you must first invent the universe."""
words_in_sagan_quote = word_tokenize(sagan_quote)
nltk.pos_tag(words_in_sagan_quote)
```

```
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
[('If', 'IN'),
 ('you', 'PRP'),
 ('wish', 'VBP'),
 ('to', 'TO'),
 ('make', 'VB'),
 ('an', 'DT'),
 ('apple', 'NN'),
 ('pie', 'NN'),
 ('from', 'IN'),
 ('scratch', 'NN'),
 (',', ','),
 ('...', ':'),
 ('you', 'PRP'),
 ('must', 'MD'),
 ('first', 'VB'),
 ('invent', 'VB'),
 ('the', 'DT'),
 ('universe', 'NN'),
 ('.', '.')]
```

```
import nltk
nltk.download('averaged_perceptron_tagger')
```

```
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
True
```

```
import nltk
nltk.download('tagsets')

nltk.download('tagsets_json')
nltk.help.upenn_tagset()
```

```
    hankerin' alleging veering capping approaching traveling besieging
    encrypting interrupting erasing wincing ...
VBN: verb, past participle
    multihulled dilapidated aerosolized chaired languished panelized used
    experimented flourished imitated reunifed factored condensed sheared
    unsettled primed dubbed desired ...
VBP: verb, present tense, not 3rd person singular
    predominate wrap resort sue twist spill cure lengthen brush terminate
    appear tend stray glisten obtain comprise detest tease attract
    emphasize mold postpone sever return wag ...
VBZ: verb, present tense, 3rd person singular
    bases reconstructs marks mixes displeases seals carps weaves snatches
    slumps stretches authorizes smolders pictures emerges stockpiles
    seduces fizzes uses bolsters slaps speaks pleads ...
WDT: WH-determiner
    that what whatever which whichever
WP: WH-pronoun
    that what whatever whatsoever which who whom whosoever
WP$: WH-pronoun, possessive
    whose
WRB: Wh-adverb
    how however whence whenever where whereby whereever wherein whereof why
``: opening quotation mark
    ` ``
[nltk_data] Downloading package tagsets to /root/nltk_data...
[nltk_data]   Package tagsets is already up-to-date!
```

```
jabberwocky_excerpt = """
... 'Twas brillig, and the slithy toves did gyre and gimble in the wabe:
... all mimsy were the borogoves, and the mome raths outgrabe."""
words_in_excerpt = word_tokenize(jabberwocky_excerpt)

nltk.pos_tag(words_in_excerpt)
```

```
[("'Twas", 'CD'),
 ('brillig', 'NN'),
 (',', ','),
 ('and', 'CC'),
 ('the', 'DT'),
 ('slithy', 'JJ'),
 ('toves', 'NNS'),
 ('did', 'VBD'),
 ('gyre', 'NN'),
 ('and', 'CC'),
 ('gimble', 'JJ'),
 ('in', 'IN'),
 ('the', 'DT'),
 ('wabe', 'NN'),
 (':', ':'),
 ('all', 'DT'),
 ('mimsy', 'NNS'),
 ('were', 'VBD'),
 ('the', 'DT'),
 ('borogoves', 'NNS'),
 (',', ','),
 ('and', 'CC'),
 ('the', 'DT'),
 ('mome', 'JJ'),
 ('raths', 'NNS'),
 ('outgrabe', 'RB'),
 ('.', '.')]
```

Lemmatizing

```
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
import nltk
nltk.download('wordnet')
```

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
True
```

```
lemmatizer.lemmatize("scarves")
```

```
'scarf'
```

```
string_for_lemmatizing = "The friends of DeSoto love scarves."
words = word_tokenize(string_for_lemmatizing)
words
```

```
['The', 'friends', 'of', 'DeSoto', 'love', 'scarves', '.']
```

```
lemmatized_words = [lemmatizer.lemmatize(word) for word in words]
lemmatized_words
```

```
['The', 'friends', 'of', 'DeSoto', 'love', 'scarves', '.']
```

```
lemmatizer.lemmatize("worst")
'worst'

lemmatizer.lemmatize("worst", pos="a")
 'bad'
```

```
from nltk.tokenize import word_tokenize
lotr_quote = "It's a dangerous business, Frodo, going out your door."
words_in_lotr_quote = word_tokenize(lotr_quote)
words_in_lotr_quote
  ['It',
   "'s",
   'a',
   'dangerous',
   'business',
   ',',
   'Frodo',
   ',',
   'going',
   'out',
   'your',
   'door',
   '.']
```

```
nltk.download("averaged_perceptron_tagger")
 [nltk_data] Downloading package averaged_perceptron_tagger to
 [nltk_data]     /root/nltk_data...
 [nltk_data]   Package averaged_perceptron_tagger is already up-to-
 [nltk_data]       date!
 True

lotr_pos_tags = nltk.pos_tag(words_in_lotr_quote)
lotr_pos_tags
```

```
[('It', 'PRP'),
 ("'s", 'VBZ'),
 ('a', 'DT'),
 ('dangerous', 'JJ'),
 ('business', 'NN'),
 (',', ','),
 ('Frodo', 'NNP'),
 (',', ','),
 ('going', 'VBG'),
 ('out', 'RP'),
 ('your', 'PRP$'),
 ('door', 'NN'),
 ('.', '.')]
```

```
grammar = "NP: {<DT>?<JJ>*<NN>}"
chunk_parser = nltk.RegexpParser (grammar)
tree = chunk_parser.parse(lotr_pos_tags)
tree.pretty_print()
```

```
                                        S
     ___________________________________|_____________________________________________________
    |     |    |     |     |     |     |     |     |          NP                          NP
    |     |    |     |     |     |     |     |     |    _______|___________              |
  It/PRP 's/VBZ ,/, Frodo/NNP ,/, going/VBG out/RP your/PRP$ ./. a/DT dangerous/JJ business/NN door/NN
```

```
lotr_pos_tags
```

```
[('It', 'PRP'),
 ("'s", 'VBZ'),
 ('a', 'DT'),
 ('dangerous', 'JJ'),
 ('business', 'NN'),
 (',', ','),
 ('Frodo', 'NNP'),
 (',', ','),
 ('going', 'VBG'),
 ('out', 'RP'),
 ('your', 'PRP$'),
 ('door', 'NN'),
 ('.', '.')]
```

```
grammar = r"""Chunk: {<.*>+} # Chunk everything
          }<JJ>{      # Exclude Adjectives
           """
chunk_parser = nltk.RegexpParser(grammar)
tree = chunk_parser.parse(lotr_pos_tags)
```

```
tree.pretty_print()
                                                S
                                                |
      _____________________________________________|____________
     |              Chunk                                Chunk
     |           ______|_____          ________________________|_____________________________
dangerous/JJ It/PRP 's/VBZ a/DT business/NN ,/, Frodo/NNP ,/, going/VBG out/RP your/PRP$ door/NN ./.
```

Using Named Entity Recognition (NER)

```
nltk.download("maxent_ne_chunker")
[nltk_data] Downloading package maxent_ne_chunker to
[nltk_data]     /root/nltk_data...
[nltk_data]   Unzipping chunkers/maxent_ne_chunker.zip.
True
```

```
nltk.download("words")
[nltk_data] Downloading package words to /root/nltk_data...
[nltk_data]   Unzipping corpora/words.zip.
True
```

```
tree = nltk.ne_chunk(lotr_pos_tags)
```

```
import nltk
from nltk.tokenize import word_tokenize

nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')
nltk.download('maxent_ne_chunker')
nltk.download('words')
# nltk.download('maxent_ne_chunker_tab') # Previously added, but might not be strictly necessary for this use-case
# nltk.download('punkt_tab') # Removed as it might be causing lookup confusion

# Re-define necessary variables for 'tree' within this cell
lotr_quote = "It's a dangerous business, Frodo, going out your door."
words_in_lotr_quote = word_tokenize(lotr_quote)
lotr_pos_tags = nltk.pos_tag(words_in_lotr_quote);

# Define 'tree' before its first use
tree = nltk.ne_chunk(lotr_pos_tags)
tree.pretty_print()
```

```
tree = nltk.ne_chunk (lotr_pos_tags, binary=True)
tree.pretty_print()

Iquote = """Men like Schiaparelli watched the red planet-it is odd, by-the-bye, that
for countless centuries Mars has been the star of war-but failed to
interpret the fluctuating appearances of the markings they mapped so well.
All that time the Martians must have been getting ready.
During the opposition of 1894 a great light was seen on the illuminated
part of the disk, first at the Lick Observatory, then by Perrotin of Nice,
and then by other observers. English readers heard of it first in the
issue of Nature dated August 2."""

def extract_ne(quote):
    words = word_tokenize(quote, language="english") # Changed to lowercase 'english'
    tags = nltk.pos_tag(words)
    tree = nltk.ne_chunk(tags, binary=True)
    return set("".join(i[0] for i in t.leaves()) for t in tree if hasattr(t, "label") and t.label() == "NE")

extract_ne(Iquote)
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package maxent_ne_chunker to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package maxent_ne_chunker is already up-to-date!
[nltk_data] Downloading package words to /root/nltk_data...
[nltk_data]   Package words is already up-to-date!
                                                S
   ______________________________________________|_____________________________________________
  |      |     |        |            |       |  |      |       |        |        |     |    PERSON
  |      |     |        |            |       |  |      |       |        |        |     |      |
It/PRP 's/VBZ a/DT dangerous/JJ business/NN ,/, ,/, going/VBG out/RP your/PRP$ door/NN ./. Frodo/NNP

                                                S
   ______________________________________________|_____________________________________________
  |      |     |        |            |       |  |      |       |        |        |     |      NE
  |      |     |        |            |       |  |      |       |        |        |     |      |
It/PRP 's/VBZ a/DT dangerous/JJ business/NN ,/, ,/, going/VBG out/RP your/PRP$ door/NN ./. Frodo/NNP

{'LickObservatory', 'Mars', 'Nature', 'Perrotin', 'Schiaparelli'}
```

Getting Text to Analyze

```
nltk.download("book")
from nltk.book import *
```

```
[nltk_data]     |
[nltk_data]  Done downloading collection book
*** Introductory Examples for the NLTK Book ***
Loading text1, ..., text9 and sent1, ..., sent9
Type the name of the text or sentence to view it.
Type: 'texts()' or 'sents()' to list the materials.
text1: Moby Dick by Herman Melville 1851
text2: Sense and Sensibility by Jane Austen 1811
text3: The Book of Genesis
text4: Inaugural Address Corpus
text5: Chat Corpus
text6: Monty Python and the Holy Grail
text7: Wall Street Journal
text8: Personals Corpus
text9: The Man Who Was Thursday by G . K . Chesterton 1908
```

Using a Concordance

text8.concordance("man")

```
Displaying 14 of 14 matches:
 to hearing from you all . ABLE young man seeks , sexy older women . Phone for
ble relationship . GENUINE ATTRACTIVE MAN 40 y . o ., no ties , secure , 5 ft .
ship , and quality times . VIETNAMESE MAN Single , never married , financially
ip . WELL DRESSED emotionally healthy man 37 like to meet full figured woman fo
 nth subs LIKE TO BE MISTRESS of YOUR MAN like to be treated well . Bold DTE no
eeks lady in similar position MARRIED MAN 50 , attrac . fit , seeks lady 40 - 5
eks nice girl 25 - 30 serious rship . Man 46 attractive fit , assertive , and k
 40 - 50 sought by Aussie mid 40s b / man f / ship r / ship LOVE to meet widowe
discreet times . Sth E Subs . MARRIED MAN 42yo 6ft , fit , seeks Lady for discr
woman , seeks professional , employed man , with interests in theatre , dining
 tall and of large build seeks a good man . I am a nonsmoker , social drinker ,
lead to relationship . SEEKING HONEST MAN I am 41 y . o ., 5 ft . 4 , med . bui
 quiet times . Seeks 35 - 45 , honest man with good SOH & similar interests , f
 genuine , caring , honest and normal man for fship , poss rship . S / S , S /
```

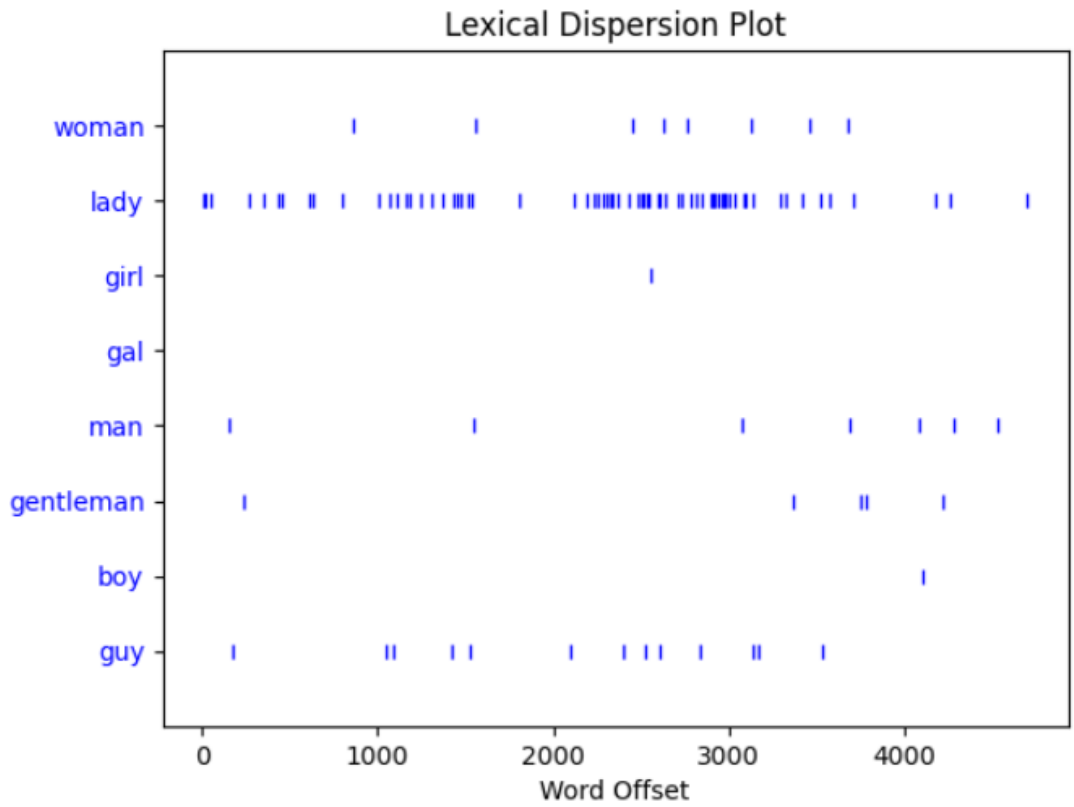
text8.concordance("woman")

```
Displaying 11 of 11 matches:
at home . Seeking an honest , caring woman , slim or med . build , who enjoys t
thy man 37 like to meet full figured woman for relationship . 48 slim , shy , S
rry . MALE 58 years old . Is there a Woman who would like to spend 1 weekend a
 other interests . Seeking Christian Woman for fship , view to rship . SWM 45 D
ALE 60 - burly beared seeks intimate woman for outings n / s s / d F / ston / P
ington . SCORPIO 47 seeks passionate woman for discreet intimate encounters SEX
le dad . 42 , East sub . 5 " 9 seeks woman 30 + for f / ship relationship TALL
personal trainer looking for married woman age open for fun MARRIED Dark guy 37
rinker , seeking slim - medium build woman who is happy in life , age open . AC
. O . TERTIARY Educated professional woman , seeks professional , employed man
 real romantic , age 50 - 65 y . o . WOMAN OF SUBSTANCE 56 , 59 kg ., 50 , fit
```

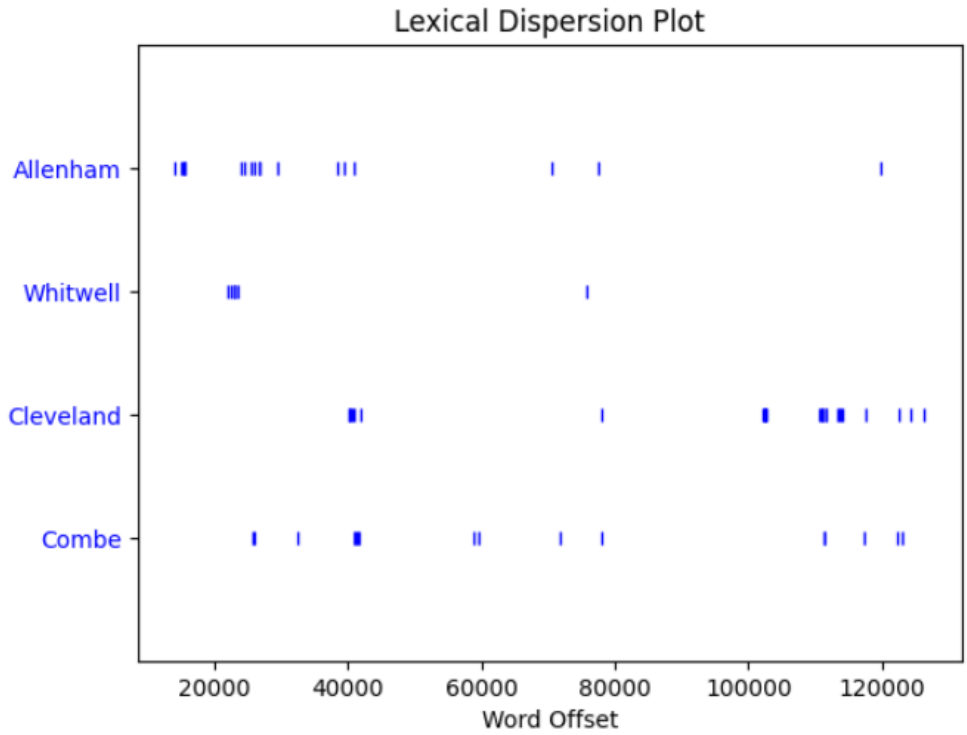
Making a Dispersion Plot

text8.dispersion_plot(
["woman", "lady", "girl", "gal", "man", "gentleman", "boy", "guy"]

)



text2.dispersion_plot(["Allenham", "Whitwell", "Cleveland", "Combe"])



Making a Frequency Distribution

from nltk import FreqDist

```
frequency_distribution = FreqDist(text8)
print(frequency_distribution)
 <FreqDist with 1108 samples and 4867 outcomes>
frequency_distribution.most_common(20)
 [(',', 539),
  ('.', 353),
  ('/', 110),
  ('for', 99),
  ('and', 74),
  ('to', 74),
  ('lady', 68),
  ('-', 66),
  ('seeks', 60),
  ('a', 52),
  ('with', 44),
  ('S', 36),
  ('ship', 33),
  ('&', 30),
  ('relationship', 29),
  ('fun', 28),
  ('in', 27),
  ('slim', 27),
  ('build', 27),
  ('o', 26)]

from nltk.corpus import stopwords
import nltk

stop_words = set(stopwords.words("english"))

meaningful_words = [
   word for word in text8 if word.casefold() not in stop_words
]
frequency_distribution.most_common(20)
```

```
[(',', 539),
 ('.', 353),
 ('/', 110),
 ('for', 99),
 ('and', 74),
 ('to', 74),
 ('lady', 68),
 ('-', 66),
 ('seeks', 60),
 ('a', 52),
 ('with', 44),
 ('S', 36),
 ('ship', 33),
 ('&', 30),
 ('relationship', 29),
 ('fun', 28),
 ('in', 27),
 ('slim', 27),
 ('build', 27),
 ('o', 26)]
```

Finding Collocations

text8.collocations()

```
would like; medium build; social drinker; quiet nights; non smoker;
long term; age open; Would like; easy going; financially secure; fun
times; similar interests; Age open; weekends away; poss rship; well
presented; never married; single mum; permanent relationship; slim
build
```

```
from nltk.stem import WordNetLemmatizer
import nltk
lemmatizer = WordNetLemmatizer()
lemmatized_words = [lemmatizer.lemmatize(word) for word in text8]
new_text = nltk.Text(lemmatized_words)
new_text.collocations()
```

```
medium build; social drinker; non smoker; quiet night; long term;
would like; age open; easy going; financially secure; Would like; fun
time; similar interest; Age open; weekend away; well presented; never
married; single mum; permanent relationship; year old; slim build
```

EXPERIMENT-2

Sentence Detection

pip install spacy

```
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (2026.2.25)
Requirement already satisfied: blis<1.4.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from thinc<8.4.0,>=8.3.12->spacy) (1.3.3)
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer<1.0.0,>=0.3.0->spacy) (8.3.2)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer<1.0.0,>=0.3.0->spacy) (1.5.4)
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer<1.0.0,>=0.3.0->spacy) (13.9.4)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer<1.0.0,>=0.3.0->spacy) (0.0.4)
Requirement already satisfied: cloudpathlib>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from weasel<2.0.0,>=1.0.0->spacy) (0.23.0)
Requirement already satisfied: smart-open>=5.2.1 in /usr/local/lib/python3.12/dist-packages (from weasel<2.0.0,>=1.0.0->spacy) (7.5.1)
Requirement already satisfied: httpx>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from weasel<2.0.0,>=1.0.0->spacy) (0.28.1)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->spacy) (3.0.3)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx>=0.24.0->weasel<2.0.0,>=1.0.0->spacy) (4.13.0)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx>=0.24.0->weasel<2.0.0,>=1.0.0->spacy) (1.0.9)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx>=0.24.0->weasel<2.0.0,>=1.0.0->spacy) (0.16.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer<1.0.0,>=0.3.0->spacy) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer<1.0.0,>=0.3.0->spacy) (2.20.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart-open>=5.2.1->weasel<2.0.0,>=1.0.0->spacy) (2.1.2)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=12.3.0->typer<1.0.0,>=0.3.0->spacy) (0.1.2)
```

```
import spacy
nlp = spacy.load("en_core_web_sm")
nlp
```

```
<spacy.lang.en.English at 0x7eb72bbc6d80>
```

```
introduction_doc = nlp("This tutorial is about Natural Language Processing in")
type(introduction_doc)
```

```
spacy.tokens.doc.Doc
```

```
[token.text for token in introduction_doc]
```

```
['This', 'tutorial', 'is', 'about', 'Natural', 'Language', 'Processing', 'in']
```

```
import pathlib
file_name = "Introduction.txt.txt"
introduction_doc = nlp(pathlib.Path(file_name).read_text(encoding="utf-8"))
print ([token.text for token in introduction_doc])
```

```
['This', 'is', 'an', 'introduction', 'to', 'Natural', 'Language', 'Processing', '.', '\n', 'import', 'pathlib', '\n', 'file_name', '=', '"', 'Introduction.txt.txt', '"', '\n'
```

Sentence Detection

```
about_text = """Gus Proto is a Python developer currently working for a London-based Fintech
company. He is interested in Natural Language Processing."""
about_doc = nlp(about_text)
sentences = list(about_doc.sents)
len(sentences)
```

```
2
```

```
for sentence in sentences:
```

```
    print(f"{sentence[:5]}...")
```

```
Gus Proto is a Python...
He is interested in Natural...
```

```
ellipsis_text = """Gus, can you, ... never mind, I forgot ... what I was saying. So, do you think we should
..."""
from spacy.language import Language
@Language.component("set_custom_boundaries")
def set_custom_boundaries (doc):
    for token in doc[:-1]:
        if token.text == "...":
            doc [token.i + 1].is_sent_start = True
    return doc
custom_nlp = spacy.load("en_core_web_sm")
custom_nlp.add_pipe("set_custom_boundaries", before="parser")
```

```
set_custom_boundaries
def set_custom_boundaries(doc)
```

```
<no docstring>
```

```
custom_ellipsis_doc = custom_nlp(ellipsis_text)
custom_ellipsis_sentences = list(custom_ellipsis_doc.sents)
custom_ellipsis_sentences = list(custom_ellipsis_doc.sents)
for sentence in custom_ellipsis_sentences:
    print(sentence)
```

```
Gus, can you, ...
never mind, I forgot ...
what I was saying.
So, do you think we should ...
```

Tokens in spaCy

```
import spacy
nlp = spacy.load("en_core_web_sm")
about_text = (
"Gus Proto is a Python developer currently"
" working for a London-based Fintech"
" company. He is interested in learning"
" Natural Language Processing."
)
about_doc = nlp(about_text)
for token in about_doc:
    print (token, token.idx)
```

```
Gus 0
Proto 4
is 10
a 13
Python 15
developer 22
currently 32
working 42
for 50
a 54
London 56
- 62
based 63
Fintech 69
company 77
. 84
He 86
is 89
interested 92
in 103
learning 106
Natural 115
Language 123
Processing 132
. 142
```

```
print(
f'{"Text with Whitespace":22}'
f'{"Is Alphanumeric?":15}'
f'{"Is Punctuation?":18}'
f'{"Is Stop Word?"}')
```

```
Text with Whitespace  Is Alphanumeric?Is Punctuation?   Is Stop Word?
```

```
for token in about_doc:
    print(
    f'{str(token.text_with_ws):22}'
    f'{str(token.is_alpha):15}'
    f'{str(token.is_punct):18}'
    f'{str(token.is_stop)}'
    )
```

```
Gus                    True          False          False
Proto                  True          False          False
is                     True          False          True
a                      True          False          True
Python                 True          False          False
developer              True          False          False
currently              True          False          False
working                True          False          False
for                    True          False          True
a                      True          False          True
London                 True          False          False
-                      False         True           False
based                  True          False          False
Fintech                True          False          False
company                True          False          False
.                      False         True           False
He                     True          False          True
is                     True          False          True
interested             True          False          False
in                     True          False          True
learning               True          False          False
Natural                True          False          False
Language               True          False          False
Processing             True          False          False
.                      False         True           False
```

```
custom_about_text = (
"Gus Proto is a Python developer currently"
" working for a London-based Fintech"
" company. He is interested in learning"
" Natural Language Processing."
)
tokens_to_print = [token.text for token in about_doc[8:15]]
print(tokens_to_print)
```

```
['for', 'a', 'London', '-', 'based', 'Fintech', 'company']
```

```
import re
import spacy
from spacy.tokenizer import Tokenizer
custom_nlp = spacy.load("en_core_web_sm")
prefix_re = spacy.util.compile_prefix_regex(custom_nlp.Defaults.prefixes)
suffix_re = spacy.util.compile_suffix_regex(custom_nlp.Defaults.suffixes)
custom_infixes = [r"@"]
```

```
infix_re = spacy.util.compile_infix_regex(
    list(custom_nlp.Defaults.infixes) + custom_infixes
)
custom_nlp.tokenizer = Tokenizer(
    custom_nlp.vocab,
    prefix_search=prefix_re.search,
    suffix_search=suffix_re.search,
    infix_finditer=infix_re.finditer,
    token_match=None,
)
custom_about_text = (
    "Gus Proto is a Python developer currently"
    " working for a London@based Fintech"
    " company. He is interested in learning"
    " Natural Language Processing."
)
custom_tokenizer_about_doc = custom_nlp(custom_about_text)
print([token.text for token in custom_tokenizer_about_doc[8:15]])
```

```
['for', 'a', 'London', '@', 'based', 'Fintech', 'company']
```

Stop Words

```
import spacy
spacy_stopwords = spacy.lang.en.stop_words.STOP_WORDS
len(spacy_stopwords)
for stop_word in list(spacy_stopwords)[:10]:
    print(stop_word)
```

```
in
too
seem
what
was
're
one
around
thus
off
```

```
import spacy
custom_about_text = (
"Gus Proto is a Python developer currently"
" working for a London-based Fintech"
" company. He is interested in learning"
" Natural Language Processing."
```

```
)
nlp = spacy.load("en_core_web_sm")
about_doc = nlp(custom_about_text)
filtered_tokens = [token.text for token in about_doc if not token.is_stop]
print(filtered_tokens)
```

```
['Gus', 'Proto', 'Python', 'developer', 'currently', 'working', 'London', '-', 'based', 'Fintech', 'company', '.', 'interested', 'learning', 'Natural', 'Language', 'Processin
```

Lemmatization

```
import spacy
nlp = spacy.load("en_core_web_sm")
conference_help_text = (
   "Gus is helping organize a developer"
   " conference on Applications of Natural Language"
   "Processing. He keeps organizing local Python meetups"
   " and several internal talks at his workplace."
)
conference_help_doc = nlp(conference_help_text)
for token in conference_help_doc:
   if str(token) != str(token.lemma_):
      print(f"{str(token):>20} : {str(token.lemma_)}")
```

```
          is : be
          He : he
        keeps : keep
   organizing : organize
     meetups : meetup
        talks : talk
```

Word Frequency

```
import spacy
from collections import Counter
nlp = spacy.load("en_core_web_sm")
complete_text = """Gus Proto is a Python developer currently working for a London-based Fintech
company. He is interested in learning Natural Language Processing.
There is a developer conference happening on 21 July 2019 in London. It is titled "Applications of
Natural Language Processing". There is a helpline number available at +44-1234567891. Gus is helping
organize it.
He keeps organizing local Python meetups and several internal talks at his workplace. Gus is also
presenting a talk. The talk will introduce the reader about "Use cases of Natural Language Processing in
Fintech".
Apart from his work, he is very passionate about music. Gus is learning to play the Piano. He has
enrolled himself in the weekend batch of Great Piano Academy.
Great Piano Academy is situated in Mayfair or the City of London and has world-class piano
instructors."""
```

```
complete_doc = nlp(complete_text)
words = [
    token.text
    for token in complete_doc
    if not token.is_stop and not token.is_punct
]
print(Counter(words).most_common(5))
```

```
[('Gus', 4), ('\n', 4), ('London', 3), ('Natural', 3), ('Language', 3)]
```

```
Counter(
[token.text for token in complete_doc if not token.is_punct]
).most_common(5)
```

```
[('is', 10), ('a', 5), ('in', 5), ('Gus', 4), ('\n', 4)]
```

Part-of-Speech Tagging

```
import spacy
nlp = spacy.load("en_core_web_sm")
about_text = (
    "Gus Proto is a Python developer currently"
    " working for a London-based Fintech"
    " company. He is interested in learning" # Added missing opening quote
    " Natural Language Processing."
)
about_doc = nlp(about_text)
for token in about_doc:
    print(
        f"""TOKEN: {str(token)}
TAG: {str(token.tag_):10} POS: {token.pos_}
EXPLANATION: {spacy.explain(token.tag_)}"""
    )
```

```
TOKEN: Gus
TAG: NNP        POS: PROPN
EXPLANATION: noun, proper singular
TOKEN: Proto
TAG: NNP        POS: PROPN
EXPLANATION: noun, proper singular
TOKEN: is
TAG: VBZ        POS: AUX
EXPLANATION: verb, 3rd person singular present
TOKEN: a
TAG: DT         POS: DET
EXPLANATION: determiner
TOKEN: Python
TAG: NNP        POS: PROPN
EXPLANATION: noun, proper singular
TOKEN: developer
TAG: NN         POS: NOUN
EXPLANATION: noun, singular or mass
TOKEN: currently
TAG: RB         POS: ADV
EXPLANATION: adverb
TOKEN: working
TAG: VBG        POS: VERB
EXPLANATION: verb, gerund or present participle
```

```
nouns = []
adjectives = []
for token in about_doc:
   if token.pos_ == "NOUN":
      nouns.append(token)
   if token.pos_ == "ADJ":
      adjectives.append(token)
nouns
```

```
[developer, company]
```

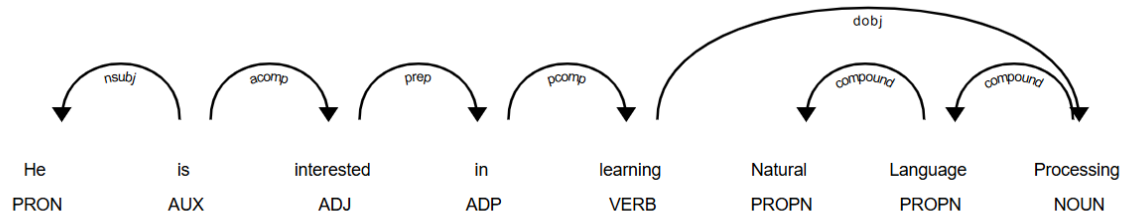
adjectives

```
[interested]
```

Visualization: Using displaCy

```
import spacy
from spacy import displacy
nlp = spacy.load("en_core_web_sm")
about_interest_text = "He is interested in learning Natural Language Processing"
```

```
about_interest_doc = nlp(about_interest_text)
# Generate HTML code for the dependency parse tree
html_code = displacy.render(about_interest_doc, style="dep", options={"distance": 120})
# Print or save the HTML code as needed
print(html_code)
```



None

Preprocessing Functions

```
import spacy
nlp = spacy.load("en_core_web_sm")
complete_text = """Gus Proto is a Python developer currently
working for a London-based Fintech company. He is
interested in learning Natural Language Processing.
There is a developer conference happening on 21 July
2019 in London. It is titled "Applications of Natural
Language Processing". There is a helpline number
available at +44-1234567891. Gus is helping organize it.
He keeps organizing local Python meetups and several
internal talks at his workplace. Gus is also presenting
a talk. The talk will introduce the reader about "Use
cases of Natural Language Processing in Fintech".
Apart from his work, he is very passionate about music.
Gus is learning to play the Piano. He has enrolled
himself in the weekend batch of Great Piano Academy.
Great Piano Academy is situated in Mayfair or the City
of London and has world-class piano instructors."""
complete_doc = nlp(complete_text)
def is_token_allowed(token):
    return bool(
        token
        and str(token).strip()
        and not token.is_stop
        and not token.is_punct
    )
def preprocess_token(token):
    return token.lemma_.strip().lower()
complete_filtered_tokens = [
    preprocess_token(token)
```

```
    for token in complete_doc
    if is_token_allowed(token)
]
print(complete_filtered_tokens)
```

```
['gus', 'proto', 'python', 'developer', 'currently', 'work', 'london', 'base', 'fintech', 'company', 'interested', 'learn', 'natural', 'language', 'processing', 'developer'
```

Rule-Based Matching Using spaCy

```
import spacy
nlp = spacy.load("en_core_web_sm")
about_text = (
    "Gus Proto is a Python developer currently"
    " working for a London-based Fintech"
    " company. He is interested in learning"
    " Natural Language Processing."
)
about_doc = nlp(about_text)
from spacy.matcher import Matcher
matcher = Matcher(nlp.vocab)
def extract_full_name(nlp_doc):
    pattern = [{"POS": "PROPN"}, {"POS": "PROPN"}]
    matcher.add("FULL_NAME", [pattern])
    matches = matcher(nlp_doc)
    for _,start, end in matches:
        span = nlp_doc[start:end]
        yield span.text
# Print the extracted full name
print(next(extract_full_name(about_doc)))
```

```
Gus Proto
```

Dependency Parsing Using spaCy

```
import spacy
nlp = spacy.load("en_core_web_sm")
one_line_about_text = (
"Gus Proto is a Python developer"
" currently working for a London-based Fintech company"
)
one_line_about_doc = nlp(one_line_about_text)
# Extract children of `developer
print([token.text for token in one_line_about_doc[5].children])
# Extract previous neighboring node of `developer
print(one_line_about_doc[5].nbor(-1))
# Extract next neighboring node of ` developer
print(one_line_about_doc[5].nbor())
# Extract all tokens on the left of `developer
print([token.text for token in one_line_about_doc[5].lefts])
```

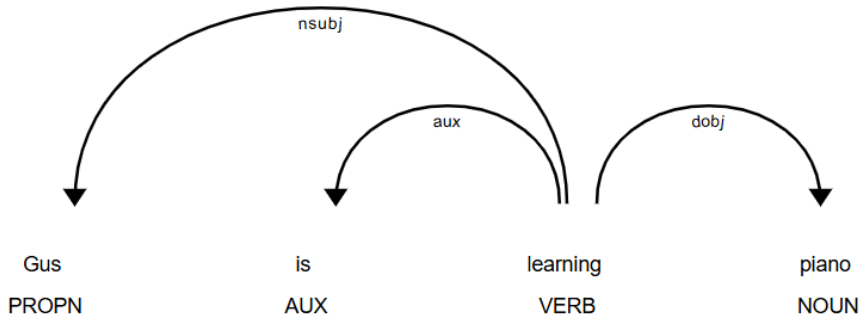
```
# Extract tokens on the right of `developer
print([token.text for token in one_line_about_doc[5].rights])
# Print subtree of`developer
print(list(one_line_about_doc[5].subtree))
```

```
['a', 'Python', 'working']
Python
currently
['a', 'Python']
['working']
[a, Python, developer, currently, working, for, a, London, -, based, Fintech, company]
```

```
import spacy
nlp = spacy.load("en_core_web_sm")
piano_text = "Gus is learning piano"
piano_doc = nlp(piano_text)
for token in piano_doc:
    print(
        f"""
TOKEN: {token.text}
{token.tag_ =}
{token.head.text =}
{token.dep_ =}"""
    )
```

```
TOKEN: Gus
token.tag_ ='NNP'
token.head.text ='learning'
token.dep_ ='nsubj'

TOKEN: is
token.tag_ ='VBZ'
token.head.text ='learning'
token.dep_ ='aux'

TOKEN: learning
token.tag_ ='VBG'
token.head.text ='learning'
token.dep_ ='ROOT'

TOKEN: piano
token.tag_ ='NN'
token.head.text ='learning'
token.dep_ ='dobj'
```

```
displacy.serve(piano_doc, style="dep")
```




```
import spacy

nlp = spacy.load("en_core_web_sm")
one_line_about_text = (
"Gus Proto is a Python developer"
" currently working for a London-based Fintech company"
)
one_line_about_doc = nlp(one_line_about_text)
# Extract children of `developer
print([token.text for token in one_line_about_doc[5].children])
# Extract previous neighboring node of `developer
print(one_line_about_doc[5].nbor(-1))
# Extract next neighboring node of ` developer
print(one_line_about_doc[5].nbor())
# Extract all tokens on the left of `developer
print([token.text for token in one_line_about_doc[5].lefts])
# Extract tokens on the right of `developer
print([token.text for token in one_line_about_doc[5].rights])
# Print subtree of`developer
print(list(one_line_about_doc[5].subtree))
```

```
['a', 'Python', 'working']
Python
currently
['a', 'Python']
['working']
[a, Python, developer, currently, working, for, a, London, -, based, Fintech, company]
```

Named-Entity Recognition

```
import spacy
nlp = spacy.load("en_core_web_sm")
piano_class_text = (
   "Great Piano Academy is situated"
   " in Mayfair or the City of London and has"
   "world-class piano instructors."
```

```
)
piano_class_doc = nlp(piano_class_text)
for ent in piano_class_doc.ents:
    print(
        f"""
{ent.text ={}
{ent.start_char ={}
{ent.end_char ={}
{ent.label_ ={}
spacy.explain('{ent.label_}') = {spacy.explain(ent.label_)}"""
    )
```

```
ent.text ='Great Piano Academy'
ent.start_char =0
ent.end_char =19
ent.label_ ='ORG'
spacy.explain('ORG') = Companies, agencies, institutions, etc.

ent.text ='Mayfair'
ent.start_char =35
ent.end_char =42
ent.label_ ='FAC'
spacy.explain('FAC') = Buildings, airports, highways, bridges, etc.

ent.text ='the City of London'
ent.start_char =46
ent.end_char =64
ent.label_ ='GPE'
spacy.explain('GPE') = Countries, cities, states
```

```
displacy.serve(piano_class_doc, style="ent")
```

Great Piano Academy ORG is situated in Mayfair FAC or the City of London GPE and hasworld-class piano instructors.

```
survey_text = """Out of 5 people surveyed, James Robert,
Julie Fuller and Benjamin Brooks like
apples. Kelly Cox and Matthew Evans
like oranges."""
def replace_person_names(token):
    if token.ent_iob != 0 and token.ent_type_ == "PERSON":
        return "[REDACTED] "
    return token.text_with_ws
def redact_names(nlp_doc):
    with nlp_doc.retokenize() as retokenizer:
        for ent in nlp_doc.ents:
            retokenizer.merge(ent)
        tokens = map(replace_person_names, nlp_doc)
```

```
    return "".join(tokens)
survey_doc = nlp(survey_text)
print(redact_names(survey_doc))
```

```
Out of 5 people surveyed, [REDACTED] [REDACTED] ,
[REDACTED] [REDACTED] and [REDACTED] [REDACTED] like
apples. [REDACTED] [REDACTED] and [REDACTED] [REDACTED]
like oranges.
```

EXPERIMENT-3
One-Hot Encoding

```
import numpy as np
# Step 1: Convert Text to Lower case
text = "My Name is AKilesh"
text_lower = text.lower()
# Step 2: Tokenize the text
tokens = text_lower.split()
# Step 3: Get unique words
unique_words = set (tokens)
# Step 4: Sort the word List
sorted_words = sorted(unique_words)
# Step 5: Get the integer/position of the words
word_to_index = {word: index for index, word in enumerate(sorted_words)}
# Step 6 and 7: Create one-hot vectors and matrix
one_hot_matrix = np.zeros((len(tokens), len(unique_words)))
for i, token in enumerate(tokens):
    word_index = word_to_index[token]
    one_hot_matrix[i, word_index] = 1
# Display the results
print("Original Text:", text)
print("One-Hot Matrix:")
print(one_hot_matrix)
```

```
Original Text: My Name is AKilesh
One-Hot Matrix:
[[0. 0. 1. 0.]
 [0. 0. 0. 1.]
 [0. 1. 0. 0.]
 [1. 0. 0. 0.]]
```

Bag of Words

```
from sklearn.feature_extraction.text import CountVectorizer
# Example Sentences
sentence1 = "My Name is AKilesh"
sentence2 = "I am a student at AVV chennai"
# Step 1: Convert to Lowercase
sentence1 = sentence1.lower()
sentence2 = sentence2.lower()
#
#
# Step 2: Remove special characters and stopwords (optional)
# In this example, we'll skip this step for simplicity.
# Step 3: Create a list of sentences
```

```
corpus = [sentence1, sentence2]
# Step 4: Use CountVectorizer to create Bag of Words model
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(corpus)
# Display the vocabulary and the Bag of Words matrix
print("Vocabulary:", vectorizer.get_feature_names_out())
print("Bag of Words Matrix:")
print(X.toarray())
```

```
Vocabulary: ['akilesh' 'am' 'at' 'avv' 'chennai' 'is' 'my' 'name' 'student']
Bag of Words Matrix:
[[1 0 0 0 0 1 1 1 0]
 [0 1 1 1 1 0 0 0 1]]
```

```
import pandas as pd
import numpy as nd
# Construct a small corpus
corpus = ['data science is one of the most important fields of science',
'this is one of the best data science courses',
'data scientists analyze data']
# Create a word set for the corpus
words_set = set()
for doc in corpus:
    words = doc.split() # Fixed: Changed from split('') to split() for proper word tokenization
    words_set = words_set.union(set(words))
print('Number of words in the corpus:', len(words_set))
print('The words in the corpus:\n', words_set)
# Computing Term Frequency (TF)
n_docs = len(corpus)
n_words_set = len(words_set)
df_tf = pd.DataFrame(np.zeros((n_docs, n_words_set)), columns=list(words_set))
# Compute Term Frequency (TF)
for i in range(n_docs):
    words = corpus[i].split(' ') # Words in the document
    for w in words:
        df_tf.loc[i, w] = df_tf.loc[i, w] + (1 / len(words)) # Fixed: Using .loc to avoid ChainedAssignmentError
print("Term Frequency (TF):")
print(df_tf)
# Computing Inverse Document Frequency (IDF)
print("\nInverse Document Frequency (IDF):")
idf = {}
for w in words_set: # Fixed: Corrected typo from words _set to words_set
    k = 0 # number of documents in the corpus that contain this word
    for i in range(n_docs):
        if w in corpus[i].split():
            k += 1
    idf[w] = np.log10(n_docs / k)
```

```
    print(f'{w:>15}: {idf[w]:>10}.') # Fixed: Removed unclosed parenthesis
# Putting it Together: Computing TF-IDF
df_tf_idf = df_tf.copy()
for w in words_set:
    for i in range(n_docs):
        df_tf_idf.loc[i, w] = df_tf.loc[i, w] * idf[w] # Fixed: Using .loc to avoid ChainedAssignmentError
print("\nTF-IDF:")
print(df_tf_idf)
# Removed: 'localhost:8888/notebooks/Downloads/NLP' was not valid Python code
```

```
Number of words in the corpus: 14
The words in the corpus:
 {'best', 'science', 'is', 'courses', 'the', 'analyze', 'scientists', 'this', 'im
Term Frequency (TF):
       best   science        is   courses       the  analyze  scientists  \
0  0.000000  0.181818  0.090909  0.000000  0.090909     0.00        0.00
1  0.111111  0.111111  0.111111  0.111111  0.111111     0.00        0.00
2  0.000000  0.000000  0.000000  0.000000  0.000000     0.25        0.25

       this  important    fields      most        of       one      data
0  0.000000   0.090909  0.090909  0.090909  0.181818  0.090909  0.090909
1  0.111111   0.000000  0.000000  0.000000  0.111111  0.111111  0.111111
2  0.000000   0.000000  0.000000  0.000000  0.000000  0.000000  0.500000

Inverse Document Frequency (IDF):
           best: 0.47712125471966244.
        science: 0.17609125905568124.
             is: 0.17609125905568124.
        courses: 0.47712125471966244.
            the: 0.17609125905568124.
        analyze: 0.47712125471966244.
     scientists: 0.47712125471966244.
           this: 0.47712125471966244.
      important: 0.47712125471966244.
         fields: 0.47712125471966244.
           most: 0.47712125471966244.
             of: 0.17609125905568124.
            one: 0.17609125905568124.
           data:        0.0.

TF-IDF:
       best   science        is   courses       the  analyze  scientists  \
0  0.000000  0.032017  0.016008  0.000000  0.016008  0.00000     0.00000
1  0.053013  0.019566  0.019566  0.053013  0.019566  0.00000     0.00000
2  0.000000  0.000000  0.000000  0.000000  0.000000  0.11928     0.11928

       this  important    fields      most        of       one  data
0  0.000000   0.043375  0.043375  0.043375  0.032017  0.016008   0.0
1  0.053013   0.000000  0.000000  0.000000  0.019566  0.019566   0.0
2  0.000000   0.000000  0.000000  0.000000  0.000000  0.000000   0.0
```

WORD2VEC

pip install gensim

```
Requirement already satisfied: gensim in /usr/local/lib/python3.12/dist-packages (4.4.0)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.1)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.1.2)
```

```
import gensim
from gensim.models import KeyedVectors
import gensim.downloader as api
# DownLoad the pre-trained Word2Vec model from Google (takes some time)
# This downloads a large file, so be patient.
word2vec_model_path = api.load("word2vec-google-news-300", return_path=True)
# Load the Word2Vec model using gensim
word2vec_model = gensim.models.KeyedVectors.load_word2vec_format(word2vec_model_path,
binary=True)
# Example: Get the word embedding for the word "king"
word_embedding = word2vec_model["king"]
# Print the dimensionality of the word embedding
print("Dimensionality of word embedding:", len(word_embedding))
# Example: Get the most similar words to "king"
similar_words = word2vec_model.most_similar("king")
print("Words most similar to 'king':", similar_words)
# Example: Calculate the similarity between two words
similarity_score = word2vec_model.similarity("king", "queen")
print("Similarity between 'king' and 'queen':", similarity_score)
# Example: Calculate the vector representing the combination of words "king" and "man" minus
result_vector = word2vec_model.most_similar(positive=["king", "man"], negative=["woman"], topn=1)
print("Vector representation of 'king- man + woman':", result_vector)
```

```
[==================================================] 100.0% 1662.8/1662.8MB downloaded
Dimensionality of word embedding: 300
Words most similar to 'king': [('kings', 0.7138045430183411), ('queen', 0.6510956883430481), ('monarch', 0.6413194537162781), ('crown_prince', 0.6204220056533813), ('prince',
Similarity between 'king' and 'queen': 0.6510957
Vector representation of 'king- man + woman': [('kings', 0.6490575075149536)]
```

pip install spacy

```
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer<1.0.0,>=0.3.0->spacy) (8.3.2)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer<1.0.0,>=0.3.0->spacy) (1.5.4)
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer<1.0.0,>=0.3.0->spacy) (13.9.4)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer<1.0.0,>=0.3.0->spacy) (0.0.4)
Requirement already satisfied: cloudpathlib>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from weasel<2.0.0,>=1.0.0->spacy) (0.23.0)
Requirement already satisfied: smart-open>=5.2.1 in /usr/local/lib/python3.12/dist-packages (from weasel<2.0.0,>=1.0.0->spacy) (7.5.1)
Requirement already satisfied: httpx>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from weasel<2.0.0,>=1.0.0->spacy) (0.28.1)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->spacy) (3.0.3)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx>=0.24.0->weasel<2.0.0,>=1.0.0->spacy) (4.13.0)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx>=0.24.0->weasel<2.0.0,>=1.0.0->spacy) (1.0.9)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx>=0.24.0->weasel<2.0.0,>=1.0.0->spacy) (0.16.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer<1.0.0,>=0.3.0->spacy) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer<1.0.0,>=0.3.0->spacy) (2.20.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart-open>=5.2.1->weasel<2.0.0,>=1.0.0->spacy) (2.1.2)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=12.3.0->typer<1.0.0,>=0.3.0->spacy) (0.1.2)
```

!python -m spacy download en_core_web_md

```
Collecting en-core-web-md==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_md-3.8.0/en_core_web_md-3.8.0-py3-none-any.whl (33.5 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 33.5/33.5 MB 15.3 MB/s eta 0:00:00
Installing collected packages: en-core-web-md
Successfully installed en-core-web-md-3.8.0
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_md')
```

GLOVE

```
import spacy
# Load the pre-trained GLoVe model
nlp = spacy.load("en_core_web_md")
# Test sentences
sentence1 = "I love natural language processing."
sentence2 = "GloVe is a powerful word embedding technique."
# Tokenize and get word embeddings for each sentence
doc1 = nlp(sentence1)
doc2 = nlp(sentence2) # Fixed: Changed n1p to nlp
# Print word embeddings for each word in the sentences
print("Word embeddings for sentence 1:")
for token in doc1:
    print(f"{token.text}: {token.vector}") # Fixed: IndentationError
print("\nWord embeddings for sentence 2:")
for token in doc2:
    print(f"{token.text}: {token.vector}") # Fixed: IndentationError
```

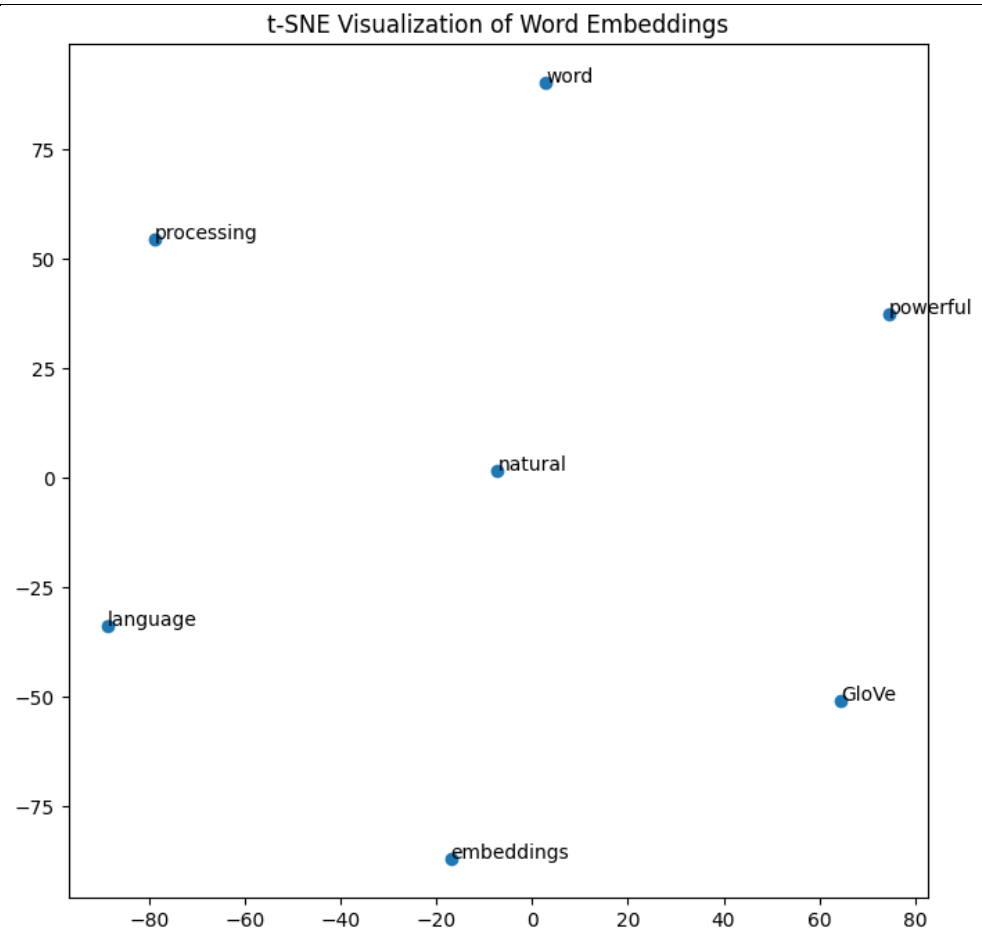
```
Word embeddings for sentence 1:
I: [-8.3712e-01 -4.0632e-01 -2.4202e-01 -3.7719e-01  5.5611e-03  2.9415e-01
 -2.1647e-01 -5.5052e-01 -8.1881e-02  1.5943e+00 -1.6071e-01  3.6091e-02
  2.3530e-01  7.5097e-02 -7.8328e-04 -1.1717e-01  1.5957e-01  2.6054e-01
 -1.3377e-01 -1.6710e-01  6.2175e-01  3.7112e-01  2.1664e-01 -2.3209e-01
 -8.2718e-02 -4.8991e-01 -2.6890e-01 -2.8994e-01  3.8026e-01 -1.7523e-01
  1.2028e-01  4.9040e-01 -1.7865e-01  1.2088e-01 -7.1101e-02  1.4119e-02
  1.4036e-01  8.7899e-02 -2.0833e-01  7.2154e-01 -3.2725e-01 -3.2122e-02
 -2.6132e-01  2.5442e-01  2.4288e-01 -1.5638e-01  3.1056e-01  3.6157e-01
 -5.4107e-01  4.2562e-01  5.0322e-02  3.1541e-01 -1.7380e-01  1.5163e-01
 -2.8188e-01  1.6727e-01  5.2491e-03 -1.8264e-01  3.1250e-01 -3.9406e-01
 -2.2052e-01  1.6621e-01 -2.4805e-01  1.3944e-01  3.9975e-01 -2.6014e-01
 -1.0253e-01  3.4850e-02 -1.0421e-01 -1.6384e-01 -1.4500e-02  1.3267e-01
  5.5815e-01 -1.5814e-01 -6.0617e-01  3.1015e-01 -9.9910e-02 -1.2486e-01
Word embeddings for sentence 2:
GloVe: [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
is: [-6.0053e-01  1.8838e-01 -4.0993e-01  3.2250e-01  7.0322e-02  1.6176e-01
  3.6945e-01 -4.1277e-01 -3.9160e-02  2.2285e+00  2.5742e-01 -2.3096e-01
 -7.2397e-02 -5.7500e-02 -3.1920e-01 -2.7190e-01 -2.6441e-01  1.3977e+00
 -1.4000e-01 -3.3425e-01 -5.5235e-01 -3.3982e-01 -1.8204e-01 -7.0568e-01
 -2.7514e-01  1.3087e-01  1.9301e-01  5.7467e-02 -2.7035e-02 -8.3781e-01
 -1.3592e-01 -3.5228e-01 -3.0606e-02 -2.2015e-01 -3.1897e-01 -1.5749e-02
  1.2286e-01 -2.4969e-01 -2.3830e-01 -1.6646e-01  2.8786e-01 -6.8925e-02
```

```
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
import numpy as np # Changed ud to np for consistency and correctness

# Define a list of strings as your sample text
x = ['text', 'the', 'leader', 'prime', 'natural', 'language']

# Create the tokenizer
tokenizer = Tokenizer()
tokenizer.fit_on_texts(x)

# Number of unique words in the dictionary
print("Number of unique words in the dictionary =", len(tokenizer.word_index))
print("Dictionary is =", tokenizer.word_index)

# Function to create an embedding matrix for the vocabulary
```

```
def embedding_for_vocab(filepath, word_index, embedding_dim):
    vocab_size = len(word_index) + 1  # Corrected indentation
    # Adding again 1 because of reserved 0 index
    embedding_matrix_vocab = np.zeros((vocab_size, embedding_dim))
    try:
        with open(filepath, encoding="utf8") as f:
            for line in f:
                word, *vector = line.split()
                if word in word_index: # Corrected 'word index' to 'word_index'
                    idx = word_index[word]
                    # Ensure the vector has enough elements before slicing
                    if len(vector) >= embedding_dim:
                        embedding_matrix_vocab[idx] = np.array(vector, dtype=np.float32)[:embedding_dim] #
Corrected slicing and variable
                    else:
                        # Handle cases where vector is too short (e.g., fill with zeros or skip)
                        print(f"Warning: Vector for '{word}' is too short. Skipping or padding with zeros.")

    except FileNotFoundError:
        print(f"Error: GloVe file '{filepath}' not found. Please ensure it's in the correct directory.")
        # Return an empty matrix or handle the error as appropriate for your application
        return np.zeros((vocab_size, embedding_dim))

    return embedding_matrix_vocab

# Define the embedding dimension
embedding_dim = 100

# Generate the embedding matrix for the given GloVe file
# Note: This will likely fail unless 'glove.6B.100d.txt' is downloaded and in the correct path
embedding_matrix_vocab = embedding_for_vocab('glove.6B.100d.txt', tokenizer.word_index,
embedding_dim) # Completed function call

# Print the dense vector for the first word in the vocabulary
# This assumes the embedding matrix was successfully created and populated
if embedding_matrix_vocab.shape[0] > 1:
    first_word_in_vocab = list(tokenizer.word_index.keys())[0]
    first_word_id = tokenizer.word_index[first_word_in_vocab]
    print(f"\nDense vector for '{first_word_in_vocab}':\n{embedding_matrix_vocab[first_word_id]}")
else:
    print("Embedding matrix is empty or not properly initialized.")
```

```
Number of unique words in the dictionary = 6
Dictionary is = {'text': 1, 'the': 2, 'leader': 3, 'prime': 4, 'natural': 5, 'language': 6}
Error: GloVe file 'glove.6B.100d.txt' not found. Please ensure it's in the correct directory.

Dense vector for 'text':
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0.]
```

```
import spacy
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE
import numpy as np # Changed 'up' to 'np'
# Load the pre-trained GloVe model
nlp = spacy.load("en_core_web_md")
# List of words for visualization
words_to_visualize = ["natural", "language", "processing", "GloVe", "powerful", "word", "embeddings"]
# Get word embeddings for the selected words
word_vectors = [nlp(word).vector for word in words_to_visualize]
# Convert word_vectors to a NumPy array
word_vectors_array = np.array(word_vectors)
# Ensure perplexity is less than the number of samples
perplexity = min(30, len(word_vectors_array) - 1)
# Apply t-SNE to reduce dimensionality to 2D
tsne = TSNE(n_components=2, perplexity=perplexity, random_state=42)
word_vectors_2d = tsne.fit_transform(word_vectors_array)
# Plot the words in 2D space
plt.figure(figsize=(8, 8))
plt.scatter(word_vectors_2d[:, 0], word_vectors_2d[:, 1])
# Annotate each point with the corresponding word
for i, word in enumerate(words_to_visualize):
    plt.annotate(word, (word_vectors_2d[i, 0], word_vectors_2d[i, 1])) # Fixed indentation
plt.title("t-SNE Visualization of Word Embeddings")
plt.show()
```



FAST TEXT

```
!pip install gensim
import nltk
from nltk.tokenize import word_tokenize
from gensim.models import FastText

# Download the 'punkt_tab' resource if not already downloaded
# This is necessary for word_tokenize to function correctly
try:
    nltk.data.find('tokenizers/punkt_tab')
except LookupError:
    nltk.download('punkt_tab')

# Dummy sentences
sentences = [
    "This is a sample sentence for FastText.",
    "FastText is a powerful word embedding model.",
    "We can train FastText on custom sentences too.",
    "Creating a simple example for demonstration.",
    "Let's check the similarity of some words."
]
# Tokenize the sentences
```

```
tokenized_sentences = [word_tokenize(sentence.lower()) for sentence in sentences]
# Train FastText model
model = FastText(
    sentences=tokenized_sentences,
    vector_size=128,
    window=5,
    min_count=1, # Adjust min_count as needed
    workers=4,
    epochs=10,
    sg=1,
    seed=42
)
# Save the trained model
model.save('dummy_fasttext_model')
# Access the word vectors
ftext = model.wv
# Check the number of unique words in the model
print("Number of unique words:", len(ftext.index_to_key))
# Check the vector size
print("Vector size:", ftext.vector_size)
# Get the vector for a specific word
word_vector = ftext["fasttext"]
print("Vector for 'fasttext':", word_vector)
# Find similar words
similar_words_mantap = ftext.similar_by_word("mantap", topn=5)
print("Similar words to 'mantap':", similar_words_mantap)
similar_words_bagus = ftext.similar_by_word("bagus", topn=5)
print("Similar words to 'bagus':", similar_words_bagus)
```

```
Requirement already satisfied: gensim in /usr/local/lib/python3.12/dist-packages (4.4.0)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.1)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.1.2)
Number of unique words: 31
Vector size: 128
Vector for 'fasttext': [-5.51286794e-04  4.71519510e-04  1.40971912e-04  2.25756690e-03
  4.95882647e-04  5.73452096e-04 -5.66022471e-04 -7.51891115e-04
  7.35156762e-04 -9.59720521e-04  2.36583335e-04  1.08922104e-04
 -2.16032015e-04 -9.02636210e-04  8.23095674e-04  1.23100157e-03
 -1.52132707e-03  1.67099410e-03 -1.51669316e-03  2.14113868e-04
  2.46162497e-04 -1.23399997e-03  1.74106914e-04 -5.27375552e-04
  1.24393078e-03 -4.13229573e-04 -9.32499010e-04 -1.73430971e-03
  1.24169965e-05 -9.87429594e-05 -7.39470488e-05 -7.31765816e-04
  8.21911206e-04  8.13115446e-04  5.54838276e-04 -1.68513355e-03
  3.78581259e-04 -3.27800633e-04  1.74142746e-03  4.84747201e-04
  1.84382414e-04 -1.04627230e-04 -7.13720685e-04 -4.84464486e-04
 -8.27850308e-04  3.63124476e-04  1.26989296e-04 -1.50212087e-04
  1.57256902e-04  1.34926639e-04  6.92375295e-04 -1.41455163e-03
 -2.15040112e-04 -1.11658883e-03  4.58740309e-04  1.05737697e-03
 -5.11300459e-04 -1.82399555e-04 -2.03541655e-04 -1.87187956e-03
 -1.02245726e-03 -5.28043893e-04  2.24034466e-05 -1.15954215e-04
 -1.16094845e-04  1.54837340e-04 -5.38311433e-04 -2.04020087e-03
  5.52054727e-04  7.16601557e-04  3.86185493e-05 -5.68291580e-04
 -4.09246684e-04 -1.51489815e-03 -1.13597652e-03 -1.49936799e-03
  1.62665220e-03 -1.73495535e-03  8.65596870e-04  1.31256774e-03
 -1.64572673e-04  4.44966252e-04 -3.82844766e-04 -3.86957050e-04
 -7.60963012e-05  5.91290416e-04 -1.13756210e-03  1.45305530e-03
 -8.68318661e-04  2.21522841e-05 -6.62541152e-06  1.71766873e-03
 -7.95343658e-04  5.14216139e-04 -5.78474777e-04 -5.59465843e-04]
Similar words to 'mantap': [('train', 0.1575692445039749 1), ('on', 0.146261528134346), ('of', 0.1271856730937958), ('fasttext', 0.12122941762208939), ('this', 0.101992473006
Similar words to 'bagus': [('for', 0.2131258100271225), ('a', 0.1904564648866653 4), ('too', 0.1188098341226577 8), ("'s", 0.0869427844882011 4), ('powerful', 0.0699156224727630
```

EXPERIMENT-4
N-gram model 1,2,3 gram

BUILDING LANGUAGE MODELS N gram model

```
def readData():
```

```
    data = ['This is a dog','This is a cat','I love my cat','This is my name']
    dat=[]
    for i in range(len(data)):
        for word in data[i].split():
            dat.append (word)
    print(dat)
    return dat
def createBigram(data):
    listOfBigrams = []
    bigramCounts = {}
    unigramCounts = {}
    for i in range(len(data)-1):
        # This condition `if i< len(data) - 1 and data[i+1].islower():` seems problematic for bigram creation.
        # It will only create bigrams where the second word starts with a lowercase letter.
        # Assuming the intention is to create all bigrams, the `islower()` check might be removed or
adjusted.
        # For now, I'm keeping it as is, but it's worth reviewing.
        if i < len(data) - 1:
            listOfBigrams.append((data[i], data[i + 1]))
            if (data[i], data[i+1]) in bigramCounts:
                bigramCounts[(data[i], data[i + 1])] += 1
            else:
                bigramCounts[(data[i], data[i + 1])] = 1
        if data[i] in unigramCounts:
            unigramCounts[data[i]] += 1
        else:
            unigramCounts[data[i]] = 1
    # Add the last word to unigram counts if not already counted
    if len(data) > 0 and data[-1] not in unigramCounts:
        unigramCounts[data[-1]] = 1
    elif len(data) > 0 and data[-1] in unigramCounts:
        unigramCounts[data[-1]] += 1

    return listOfBigrams, unigramCounts, bigramCounts
def calcBigramProb(listOfBigrams, unigramCounts, bigramCounts):
    listOfProb = {}
    for bigram in listOfBigrams:
        word1 = bigram[0]
        word2 = bigram[1]
        # Handle potential division by zero if unigram count is 0
        if unigramCounts.get(word1, 0) > 0:
            listOfProb[bigram] = (bigramCounts.get(bigram, 0))/(unigramCounts.get(word1))
        else:
            listOfProb[bigram] = 0 # Or handle as appropriate, e.g., NaN, skip
    return listOfProb
if __name__ == '__main__': # Fixed typo
```

```
data_words = readData() # Changed variable name to avoid conflict, fixed function call
listOfBigrams, unigramCounts, bigramCounts = createBigram(data_words) # Corrected indentation
print("\n All the possible Bigrams are ")
print(listOfBigrams)
print("\n Bigrams along with their frequency ")
print(bigramCounts)
print("\n Unigrams along with their frequency ")
print(unigramCounts)
bigramProb = calcBigramProb(listOfBigrams, unigramCounts, bigramCounts)
print("\n Bigrams along with their probability ")
print (bigramProb)
inputList="This is my cat"
splt=inputList.split()
outputProb1 = 1
bilist=[] # Fixed initialization
# bigrm=[] # This variable was defined but not used. Removed.
for i in range(len(splt) - 1):
    if i < len(splt) - 1: # Fixed missing comparison operator
        bilist.append((splt[i], splt[i + 1]))
print("\n The bigrams in given sentence are ")
print(bilist)
for i in range(len(bilist)):
    if bilist[i] in bigramProb:
        outputProb1 *= bigramProb[bilist[i]]
    else:
        outputProb1 *= 0
```

```
['This', 'is', 'a', 'dog', 'This', 'is', 'a', 'cat', 'I', 'love', 'my', 'cat', 'This', 'is', 'my', 'name']

 All the possible Bigrams are
[('This', 'is'), ('is', 'a'), ('a', 'dog'), ('dog', 'This'), ('This', 'is'), ('is', 'a'), ('a', 'cat'), ('cat', 'I'), ('I', 'love'), ('love', 'my'), ('my', 'cat'), ('cat', '

 Bigrams along with their frequency
{('This', 'is'): 3, ('is', 'a'): 2, ('a', 'dog'): 1, ('dog', 'This'): 1, ('a', 'cat'): 1, ('cat', 'I'): 1, ('I', 'love'): 1, ('love', 'my'): 1, ('my', 'cat'): 1, ('cat', 'Th

 Unigrams along with their frequency
{'This': 3, 'is': 3, 'a': 2, 'dog': 1, 'cat': 2, 'I': 1, 'love': 1, 'my': 2, 'name': 1}

 Bigrams along with their probability
{('This', 'is'): 1.0, ('is', 'a'): 0.666666666666666, ('a', 'dog'): 0.5, ('dog', 'This'): 1.0, ('a', 'cat'): 0.5, ('cat', 'I'): 0.5, ('I', 'love'): 1.0, ('love', 'my'): 1.0

 The bigrams in given sentence are
[('This', 'is'), ('is', 'my'), ('my', 'cat')]
```

```
import nltk
nltk.download('punkt_tab')
```

```
import nltk
nltk.download('reuters')
from nltk.corpus import reuters
from nltk import bigrams, trigrams
from collections import Counter, defaultdict
# Create a placeholder for model
model = defaultdict(lambda: defaultdict(lambda: 0))
# Count frequency of co-occurance
```

```
for sentence in reuters.sents():
    for w1, w2, w3 in trigrams (sentence, pad_right=True, pad_left=True):
        model[(w1, w2)][w3] += 1
# Let's transform the counts to probabilities
for w1, w2 in model:
    total_count = float(sum(model[(w1, w2)].values()))
    for w3 in model[(w1, w2)]:
        model[(w1, w2)][w3] /= total_count
dict(model["today", "the"])
```

```
[nltk_data] Downloading package reuters to /root/nltk_data...
{'public': 0.05555555555555555,
 'European': 0.05555555555555555,
 'Bank': 0.05555555555555555,
 'price': 0.1111111111111111,
 'emirate': 0.05555555555555555,
 'overseas': 0.05555555555555555,
 'newspaper': 0.05555555555555555,
 'company': 0.16666666666666666,
 'Turkish': 0.05555555555555555,
 'increase': 0.05555555555555555,
 'options': 0.05555555555555555,
 'Higher': 0.05555555555555555,
 'pound': 0.05555555555555555,
 'Italian': 0.05555555555555555,
 'time': 0.05555555555555555}
```

```
import random
# starting words
text = ["today", "the"]
sentence_finished = False
while not sentence_finished:
    # select a random probability threshold
    r = random.random()
    accumulator = 0.0
    for word in model[tuple(text[-2:])].keys():
        accumulator += model[tuple(text[-2:])][word]
    # select words that are above the probability threshold
    if accumulator >= r:
        text.append(word)
        break
    if text[-2:] == [None, None]:
        sentence_finished = True
print (' '.join([t for t in text if t]))
```

```
today the time
```

```
import numpy as np
```

```
import pandas as pd
from keras.utils import to_categorical
from keras.preprocessing.sequence import pad_sequences
from keras.models import Sequential
from keras.layers import LSTM, Dense, GRU, Embedding
from keras.callbacks import EarlyStopping, ModelCheckpoint
data_text = """The unanimous Declaration of the thirteen united States of America, When in
We hold these truths to be self-evident, that all men are created equal, that they are endo
He has refused his Assent to Laws, the most wholesome and necessary for the public good.
He has forbidden his Governors to pass Laws of immediate and pressing importance, unless su
He has refused to pass other Laws for the accommodation of large districts of people, unles:
He has called together legislative bodies at places unusual, uncomfortable, and distant fror
He has dissolved Representative Houses repeatedly, for opposing with manly firmness his inva
He has refused for a long time, after such dissolutions, to cause others to be elected; whe
He has endeavoured to prevent the population of these States; for that purpose obstructing
He has obstructed the Administration of Justice, by refusing his Assent to Laws for establi:
He has made Judges dependent on his Will alone, for the tenure of their offices, and the am
He has erected a multitude of New Offices, and sent hither swarms of Officers to harrass ou
He has kept among us, in times of peace, Standing Armies without the Consent of our legisla
He has affected to render the Military independent of and superior to the Civil power.
He has combined with others to subject us to a jurisdiction foreign to our constitution, and
For Quartering large bodies of armed troops among us:
For protecting them, by a mock Trial, from punishment
For cutting off our Trade with all parts of the world:
For imposing Taxes on us without our Consent:
for any Murders which they should comr
For depriving us in many cases, of the benefits of Trial by Jury:
For transporting us beyond Seas to be tried for pretended offences
For abolishing the free System of English Laws in a neighbouring Province, establishing the
For taking away our Charters, abolishing our most valuable Laws, and altering fundamentally
For suspending our own Legislatures, and declaring themselves invested with power to legisl
He has abdicated Government here, by declaring us out of his Protection and waging War agai
He has plundered our seas, ravaged our Coasts, burnt our towns, and destroyed the lives of
He is at this time transporting large Armies of foreign Mercenaries to compleat the works
He has constrained our fellow Citizens taken Captive on the high Seas to bear Arms against
He has excited domestic insurrections amongst us, and has endeavoured to bring on the inhab:
In every stage of these Oppressions We have Petitioned for Redress in the most humble terms
Nor have We been wanting in attentions to our Brittish brethren. We have warned them from
We, therefore, the Representatives of the united States of America, in General Congress, As"""
import re
def text_cleaner(text):
    # Lower case text
    newString = text.lower()
    newString = re.sub(r"'s\b","",newString)
    # remove punctuations
    newString = re.sub("[^a-zA-Z]", " ", newString)
```

```
    long_words=[]
    # remove short word
    for i in newString.split():
        if len(i)>=3:
            long_words.append(i)
    return (" ".join(long_words)).strip()
# preprocess the text
data_new = text_cleaner(data_text)

def create_seq(text):
    length = 30
    sequences = list()
    for i in range(length, len(text)):
        # select sequence of tokens
        seq = text[i-length:i+1]
        # store
        sequences.append(seq)
    print('Total Sequences: %d' % len(sequences))
    return sequences
# create sequences
sequences = create_seq(data_new)
```

```
Total Sequences: 2358
```

```
chars = sorted(list(set(data_new)))
mapping = dict((c, i) for i, c in enumerate(chars))
def encode_seq(seq):
    sequences = list()
    for line in seq:
        # integer encode Line
        encoded_seq = [mapping [char] for char in line]
        # store
        sequences.append (encoded_seq)
    return sequences
# encode the sequences
sequences = encode_seq(sequences)
def encode_seq(seq, mapping):
    encoded_seqs = []
    for line in seq:
        # integer encode Line
        encoded_seq = [mapping[char] for char in line]
        # store
        encoded_seqs.append(encoded_seq)
    return np.array(encoded_seqs)
```

```
[[20,
  8,
  5,
  0,
  21,
  14,
  1,
  14,
  9,
  13,
  15,
  21,
  19,
  0,
  4,
  5,
  3,
  12,
  1,
  15,
  21,
  18,
  0,
  12,
  5,
  7,
  9,
  19,
  12,
  1,
  0,
  8,
  1,
  19,
  0,
  1,
  6,
  6,
  5],
 ...]
```

```
vocab = len(mapping)
sequences = np.array(sequences)
X, y = sequences[:, :-1], sequences[:, -1]
# one hot encode y
y = to_categorical(y, num_classes=vocab)

from sklearn.model_selection import train_test_split
# vocabulary size
vocab = len(mapping)
sequences = np.array (sequences)
# create X and y
# create train and validation sets
X_tr, X_val, y_tr, y_val = train_test_split(X, y, test_size=0.1, random_state=42)
print('Train shape:', X_tr.shape, 'Val shape:', X_val.shape)
```

```
Train shape: (2122, 30) Val shape: (236, 30)
```

```
from sklearn.model_selection import train_test_split
from keras.utils import to_categorical
import numpy as up
```

```
# Assuming you have a 'mapping' variable defined elsewhere
# vocab = Len(mapping)
# Assuming you have a 'sequences' variable defined elsewhere
# sequences = np.array(sequences)
# Reshape sequences to make it 2-dimensional
sequences = sequences.reshape((len(sequences), sequences.shape[1]))
# create X and y
X, y = sequences[:, :-1], sequences[:, -1]
# one hot encode y
y = to_categorical(y, num_classes=vocab)
# create train and validation sets
X_tr, X_val, y_tr, y_val = train_test_split(X, y, test_size=0.1, random_state=42)
print('Train shape:', X_tr.shape, 'Val shape:', X_val.shape)
```

```
Train shape: (2122, 30) Val shape: (236, 30)
```

```
model = Sequential()
model.add(Embedding(vocab, 50, input_length=30, trainable=True))
model.add(GRU(150, recurrent_dropout=0.1, dropout=0.1))
model.add(Dense(vocab, activation='softmax'))
print(model.summary())
# compile the model
model.compile(loss='categorical_crossentropy', metrics=['acc'], optimizer='adam')
# fit the model
model.fit(X_tr, y_tr, epochs=100, verbose=2, validation_data=(X_val, y_val))
```

```
Epoch 73/100
67/67 - 6s - 95ms/step - acc: 0.9764 - loss: 0.1202 - val_acc: 0.3856 - val_loss: 3.7422
Epoch 74/100
67/67 - 10s - 147ms/step - acc: 0.9713 - loss: 0.1256 - val_acc: 0.3771 - val_loss: 3.7785
Epoch 75/100
67/67 - 5s - 77ms/step - acc: 0.9769 - loss: 0.1088 - val_acc: 0.3644 - val_loss: 3.8492
Epoch 76/100
67/67 - 5s - 73ms/step - acc: 0.9750 - loss: 0.1188 - val_acc: 0.3771 - val_loss: 3.8602
Epoch 77/100
67/67 - 7s - 99ms/step - acc: 0.9774 - loss: 0.1067 - val_acc: 0.3814 - val_loss: 3.8492
Epoch 78/100
67/67 - 5s - 73ms/step - acc: 0.9783 - loss: 0.1099 - val_acc: 0.3856 - val_loss: 3.9031
Epoch 79/100
67/67 - 7s - 100ms/step - acc: 0.9793 - loss: 0.1006 - val_acc: 0.3771 - val_loss: 3.9321
Epoch 80/100
67/67 - 5s - 73ms/step - acc: 0.9821 - loss: 0.1002 - val_acc: 0.3729 - val_loss: 3.9666
Epoch 81/100
67/67 - 6s - 85ms/step - acc: 0.9736 - loss: 0.1078 - val_acc: 0.3771 - val_loss: 4.0417
Epoch 82/100
67/67 - 6s - 87ms/step - acc: 0.9746 - loss: 0.1034 - val_acc: 0.3729 - val_loss: 4.0398
Epoch 83/100
67/67 - 11s - 161ms/step - acc: 0.9774 - loss: 0.0991 - val_acc: 0.3814 - val_loss: 4.0700
Epoch 84/100
67/67 - 9s - 130ms/step - acc: 0.9779 - loss: 0.1019 - val_acc: 0.3644 - val_loss: 4.0694
Epoch 85/100
```

```
Epoch 91/100
67/67 - 11s - 163ms/step - acc: 0.9840 - loss: 0.0793 - val_acc: 0.3729 - val_loss: 4.1468
Epoch 92/100
67/67 - 5s - 72ms/step - acc: 0.9835 - loss: 0.0793 - val_acc: 0.3729 - val_loss: 4.2384
Epoch 93/100
67/67 - 6s - 91ms/step - acc: 0.9783 - loss: 0.0825 - val_acc: 0.3771 - val_loss: 4.2117
Epoch 94/100
67/67 - 5s - 79ms/step - acc: 0.9821 - loss: 0.0796 - val_acc: 0.3941 - val_loss: 4.2637
Epoch 95/100
67/67 - 5s - 73ms/step - acc: 0.9877 - loss: 0.0709 - val_acc: 0.3729 - val_loss: 4.2526
Epoch 96/100
67/67 - 7s - 100ms/step - acc: 0.9811 - loss: 0.0745 - val_acc: 0.3941 - val_loss: 4.2767
Epoch 97/100
67/67 - 9s - 142ms/step - acc: 0.9821 - loss: 0.0771 - val_acc: 0.3814 - val_loss: 4.1967
Epoch 98/100
67/67 - 9s - 141ms/step - acc: 0.9807 - loss: 0.0791 - val_acc: 0.3814 - val_loss: 4.3032
Epoch 99/100
67/67 - 7s - 99ms/step - acc: 0.9873 - loss: 0.0710 - val_acc: 0.3686 - val_loss: 4.2692
Epoch 100/100
67/67 - 9s - 137ms/step - acc: 0.9821 - loss: 0.0732 - val_acc: 0.3771 - val_loss: 4.2918
<keras.src.callbacks.history.History at 0x7f4f714a40e0>
```

```
import numpy as np
from keras.preprocessing.sequence import pad_sequences

def generate_seq(model, mapping, seq_length, seed_text, n_chars):
    in_text = seed_text
    # generate a fixed number of characters
    for _ in range(n_chars):
        # encode the characters as integers
        encoded = [mapping[char] for char in in_text]
        # truncate sequences to a fixed length
        encoded = pad_sequences([encoded], maxlen=seq_length, truncating='pre')
        # predict character probabilities
        yhat_probs = model.predict(encoded, verbose=0)
        # get the index of the character with the highest probability
        yhat = np.argmax(yhat_probs)

        # reverse map integer to character
        out_char = '' # Fixed unterminated string literal
        for char_key, index_val in mapping.items():
            if index_val == yhat:
                out_char = char_key
                break
        # append to input
        in_text += out_char
    return in_text


import numpy as np
from keras.preprocessing.sequence import pad_sequences
```

```
def generate_seq(model, mapping, seq_length, seed_text, n_chars):
    in_text = seed_text
    # generate a fixed number of characters
    for _ in range(n_chars):
        # encode the characters as integers
        encoded = [mapping[char] for char in in_text]
        # truncate sequences to a fixed length
        encoded = pad_sequences([encoded], maxlen=seq_length, truncating='pre')
        # predict character probabilities
        yhat_probs = model.predict(encoded, verbose=0)
        # get the index of the character with the highest probability
        yhat = np.argmax(yhat_probs)

        # reverse map integer to character
        out_char = ''
        for char_key, index_val in mapping.items():
            if index_val == yhat:
                out_char = char_key
                break
        # append to input
        in_text += out_char
    return in_text

inp='Large armies of'
print(len(inp))
print(generate_seq(model,mapping, 50, inp.lower(),15))
```

```
15
large armies offices and sent
```

EXPERIMENT-5
Tokenization

```
import nltk
from nltk.tokenize import word_tokenize
nltk.download('punkt') # Ensure 'punkt_tab' tokenizer is downloaded
nltk.download('punkt_tab') # Download the 'punkt_tab' resource
nltk.download('averaged_perceptron_tagger_eng') # Download the POS tagger
text = word_tokenize("And now for something completely different")
tags = nltk.pos_tag(text)
print(tags)
```

```
[('And', 'CC'), ('now', 'RB'), ('for', 'IN'), ('something', 'NN'), ('completely', 'RB'), ('different', 'JJ')]
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
```

```
import nltk
nltk.download('brown')
text = nltk.Text(word.lower() for word in nltk.corpus.brown.words())
text.similar('woman')
text.similar('bought')
text.similar('over')
text.similar('the')
```

```
[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data]   Package brown is already up-to-date!
man time day year car moment world house family child country boy
state job place way war girl work word
made said done put had seen found given left heard was been brought
set got that took in told felt
in on to of and for with from at by that into as up out down through
is all about
a his this their its her an that our any all one these my in your no
some other and
```

Tagged Corpora

```
tagged_token = nltk.tag.str2tuple('fly/NN')
print(tagged_token)
print(tagged_token[0])
print(tagged_token[1])
```

```
('fly', 'NN')
fly
NN
```

```
import nltk
sent ="""
The/AT grand/JJ jury/NN commented/VBD on/IN a/AT number/NN of/IN
other/AP topics/NNS,/, AMONG/IN them/PPO the/AT Atlanta/NP and/CC
Fulton/NP-tl County/NN-tl purchasing/VBG departments/NNS which/WDT it/PPS
said/VBD/ ARE/BER well/QL operated/VBN and/CC follow/VB generally/RB
accepted/VBN practices/NNS which/WDT inure/VB to/IN the/AT best/JJT
interest/NN of/IN both/ABX governments/NNS "/"."""
tagged_tokens = [nltk.tag.str2tuple(t) for t in sent.split()]
print(tagged_tokens)
```

```
[('The', 'AT'), ('grand', 'JJ'), ('jury', 'NN'), ('commented', 'VBD'), ('on', 'IN'), ('a', 'AT'), ('number', 'NN'), ('of', 'IN'), ('other', 'AP'), ('topics/NNS,', ','), ('AMO
```

```
import nltk
nltk.download('universal_tagset') # Download the universal tagset
# Using the default Brown corpus tagset
tagged_words_default = nltk.corpus.brown.tagged_words()
print(tagged_words_default)
# Using the Universal Tagset for the Brown corpus
```

```
tagged_words_universal = nltk.corpus.brown.tagged_words(tagset='universal')
print(tagged_words_universal)
```

```
[('The', 'AT'), ('Fulton', 'NP-TL'), ...]
[('The', 'DET'), ('Fulton', 'NOUN'), ...]
[nltk_data] Downloading package universal_tagset to /root/nltk_data...
[nltk_data]   Package universal_tagset is already up-to-date!
```

```
import nltk
nltk.download('nps_chat')
nltk.download('conll2000')
nltk.download('treebank')
# Access tagged words from the NPS Chat corpus
nps_chat_tagged_words = nltk.corpus.nps_chat.tagged_words()
print(nps_chat_tagged_words)
# Access tagged words from the CoNLL 2000 corpus
con112000_tagged_words = nltk.corpus.conll2000.tagged_words()
print(con112000_tagged_words)
# Access tagged words from the Treebank corpus
treebank_tagged_words = nltk.corpus.treebank.tagged_words()
print(treebank_tagged_words)
```

```
[('now', 'RB'), ('im', 'PRP'), ('left', 'VBD'), ...]
[('Confidence', 'NN'), ('in', 'IN'), ('the', 'DT'), ...]
[('Pierre', 'NNP'), ('Vinken', 'NNP'), (',', ','), ...]
[nltk_data] Downloading package nps_chat to /root/nltk_data...
[nltk_data]   Package nps_chat is already up-to-date!
[nltk_data] Downloading package conll2000 to /root/nltk_data...
[nltk_data]   Package conll2000 is already up-to-date!
[nltk_data] Downloading package treebank to /root/nltk_data...
[nltk_data]   Package treebank is already up-to-date!
```

```
import nltk
# Access tagged words from the Brown corpus with the Universal Tagset
brown_tagged_words_universal = nltk.corpus.brown.tagged_words(tagset='universal')
print(brown_tagged_words_universal)
# Access tagged words from the Treebank corpus with the Universal Tagset
treebank_tagged_words_universal = nltk.corpus.treebank.tagged_words (tagset='universal')
print(treebank_tagged_words_universal)
```

```
[('The', 'DET'), ('Fulton', 'NOUN'), ...]
[('Pierre', 'NOUN'), ('Vinken', 'NOUN'), (',', '.'), ...]
```

```
import nltk
nltk.download('sinica_treebank')
nltk.download('indian')
nltk.download('mac_morpho')
nltk.download('conll2002') # Corrected from conl12002
```

```
nltk.download('cess_cat')
# Access tagged words from the Sinica Treebank corpus
sinica_treebank_tagged_words = nltk.corpus.sinica_treebank.tagged_words()
print(sinica_treebank_tagged_words)
# Access tagged words from the Indian corpus
indian_tagged_words = nltk.corpus.indian.tagged_words()
print(indian_tagged_words)
# Access tagged words from the Mac-Morpho corpus
mac_morpho_tagged_words = nltk.corpus.mac_morpho.tagged_words()
print(mac_morpho_tagged_words)
# Access tagged words from the CoNLL 2002 corpus
con112002_tagged_words = nltk.corpus.conll2002.tagged_words()
print(con112002_tagged_words)
# Access tagged words from the CESS-CAT corpus
cess_cat_tagged_words = nltk.corpus.cess_cat.tagged_words()
print(cess_cat_tagged_words)
```

```
[('一', 'Neu'), ('友情', 'Nad'), ('嘉珍', 'Nba'), ...]
[('মহিষের', 'NN'), ('সন্তান', 'NN'), (':', 'SYM'), ...]
[('Jersei', 'N'), ('atinge', 'V'), ('média', 'N'), ...]
[('Sao', 'NC'), ('Paulo', 'VMI'), ('(', 'Fpa'), ...]
[('El', 'da0ms0'), ('Tribunal_Suprem', 'np0000o'), ...]
[nltk_data] Downloading package sinica_treebank to /root/nltk_data...
[nltk_data]   Package sinica_treebank is already up-to-date!
[nltk_data] Downloading package indian to /root/nltk_data...
[nltk_data]   Package indian is already up-to-date!
[nltk_data] Downloading package mac_morpho to /root/nltk_data...
[nltk_data]   Package mac_morpho is already up-to-date!
[nltk_data] Downloading package conll2002 to /root/nltk_data...
[nltk_data]   Package conll2002 is already up-to-date!
[nltk_data] Downloading package cess_cat to /root/nltk_data...
[nltk_data]   Package cess_cat is already up-to-date!
```

```
from nltk.corpus import brown
brown_news_tagged = brown.tagged_words (categories='news', tagset='universal')
tag_fd = nltk.FreqDist(tag for (word, tag) in brown_news_tagged)
tag_fd.most_common()
```

```
[('NOUN', 30654),
 ('VERB', 14399),
 ('ADP', 12355),
 ('.', 11928),
 ('DET', 11389),
 ('ADJ', 6706),
 ('ADV', 3349),
 ('CONJ', 2717),
 ('PRON', 2535),
 ('PRT', 2264),
 ('NUM', 2166),
 ('X', 92)]
```

```
# Load the Brown corpus and filter it to use only the 'news' category
brown_news_tagged = nltk.corpus.brown.tagged_words (categories='news')
# Generate bigrams from the tagged words in the 'news' category
word_tag_pairs = nltk.bigrams (brown_news_tagged)
# Filter for cases where the second item in the bigram is tagged as a noun ('NOUN')
noun_preceders = [a[1] for (a, b) in word_tag_pairs if b[1] == 'NOUN']
# Create a frequency distribution of the tags that precede nouns
fdist = nltk.FreqDist(noun_preceders)
# Extract the most common tags that precede nouns
most_common_tags = [tag for (tag, _) in fdist.most_common()]
print (most_common_tags)
```

```
[]
```

```
import nltk
# Access tagged words from the Treebank corpus with the Universal Tagset
wsj = nltk.corpus.treebank.tagged_words (tagset='universal')
# Create a frequency distribution of word-tag pairs
word_tag_fd = nltk.FreqDist(wsj)
# Extract the most common words tagged as verbs
verbs = [wt[0] for (wt, _) in word_tag_fd.most_common() if wt[1] == 'VERB']
print(verbs)
```

```
['is', 'said', 'was', 'are', 'be', 'has', 'have', 'will', 'says', 'would', 'were', 'had', 'been', 'could', "'s", 'can', 'do', 'say', 'make', 'may', 'did', 'rose', 'made', 'do
```

```
import nltk

# Assuming 'wsj' contains the Wall Street Journal portion of the Treebank corpus
# If 'wsj' is not defined yet, load the Wall Street Journal portion of the Treebank corpus
wsj = nltk.corpus.treebank.tagged_words()
# Create a ConditionalFreqDist object
cfd1 = nltk.ConditionalFreqDist(wsj)
```

```
# Check the distribution of parts of speech for the word 'yield'
yield_pos_distribution = cfd1['yield'].most_common()
print("Distribution of 'yield':", yield_pos_distribution)
# Check the distribution of parts of speech for the word
cut_pos_distribution = cfd1['cut'].most_common()
print("Distribution of 'cut':", cut_pos_distribution)
```

```
Distribution of 'yield': [('VB', 28), ('NN', 20)]
Distribution of 'cut': [('VB', 12), ('VBD', 10), ('VBN', 3), ('NN', 3)]
```

```
import nltk
# Assuming 'wsj' contains the Wall Street Journal portion of the Treebank corpus
# If 'wsj' is not defined yet, load the Wall Street Journal portion of the Treebank corpus
wsj = nltk.corpus.treebank.tagged_words()
# Create a ConditionalFreqDist object by swapping word and tag positions
cfd2 = nltk.ConditionalFreqDist((tag, word) for (word, tag) in wsj)
# List words tagged as 'VBN' (past participle)
vbn_words = list(cfd2['VBN'])
print(vbn_words[:20]) # Display the first 20 words for brevity
```

```
['been', 'expected', 'made', 'compared', 'based', 'used', 'priced', 'sold', 'named', 'designed', 'held', 'fined', 'taken', 'paid', 'traded', 'increased',
```

```
import nltk
# Assuming 'wsj' contains the Wall Street Journal portion of the Treebank corpus
# If 'wsj' is not defined yet, load the Wall Street Journal portion of the Treebank corpus
wsj = nltk.corpus.treebank.tagged_words()
# Create a ConditionalFreqDist object 'cfd1'
cfd1 = nltk.ConditionalFreqDist((word, tag) for (word, tag) in wsj)
# Find words that are tagged as both 'VBD' (past tense) and 'VBN' (past participle)
words_with_vbd_and_vbn = [w for w in cfd1.conditions() if 'VBD' in cfd1[w] and 'VBN' in cfd1[w]]
print(words_with_vbd_and_vbn[:10]) # Display the first 10 words for brevity
# Find context surrounding 'kicked' as 'VBD'
idx1 = wsj.index(('kicked', 'VBD'))
print(wsj[idx1-4:idx1+1])
# Find context surrounding 'kicked' as 'VBN'
idx2 = wsj.index(('kicked', 'VBN'))
print(wsj[idx2-4:idx2+1])
```

```
['named', 'used', 'caused', 'reported', 'said', 'stopped', 'heard', 'studied', 'led', 'replaced']
[('While', 'IN'), ('program', 'NN'), ('trades', 'NNS'), ('swiftly', 'RB'), ('kicked', 'VBD')]
[('head', 'NN'), ('of', 'IN'), ('state', 'NN'), ('has', 'VBZ'), ('kicked', 'VBN')]
```

```
import nltk
def findtags (tag_prefix, tagged_text):
    cfd = nltk.ConditionalFreqDist((tag, word) for (word, tag) in tagged_text
                                   if tag.startswith(tag_prefix))
    return {tag: cfd[tag].most_common(5) for tag in cfd.conditions()}
tagged_news = nltk.corpus.brown.tagged_words (categories='news')
tagdict = findtags('NN', tagged_news)
for tag in sorted(tagdict):
```

```
    print(tag, tagdict[tag])
NN [('year', 137), ('time', 97), ('state', 88), ('week', 85), ('man', 72)]
NN$ [("year's", 13), ("world's", 8), ("state's", 7), ("nation's", 6), ("city's", 6)]
NN$-HL [("Golf's", 1), ("Navy's", 1)]
NN$-TL [("President's", 11), ("Administration's", 3), ("Army's", 3), ("League's", 3), ("University's", 3)]
NN-HL [('sp.', 2), ('problem', 2), ('Question', 2), ('cut', 2), ('party', 2)]
NN-NC [('ova', 1), ('eva', 1), ('aya', 1)]
NN-TL [('President', 88), ('House', 68), ('State', 59), ('University', 42), ('City', 41)]
NN-TL-HL [('Fort', 2), ('Mayor', 1), ('Commissioner', 1), ('City', 1), ('Oak', 1)]
NNS [('years', 101), ('members', 69), ('people', 52), ('sales', 51), ('men', 46)]
NNS$ [("children's", 7), ("women's", 5), ("men's", 3), ("janitors'", 3), ("taxpayers'", 2)]
NNS$-HL [("Dealers'", 1), ("Idols'", 1)]
NNS$-TL [("Women's", 4), ("States'", 3), ("Giants'", 2), ("Princes'", 1), ("Bombers'", 1)]
NNS-HL [('Wards', 1), ('deputies', 1), ('bonds', 1), ('aspects', 1), ('Decisions', 1)]
NNS-TL [('States', 38), ('Nations', 11), ('Masters', 10), ('Communists', 9), ('Rules', 9)]
NNS-TL-HL [('Nations', 1)]
```

```
import nltk
# Assuming 'brown_learned_text' contains the words from the 'learned' category of the Brown corpus
brown_learned_text = nltk.corpus.brown.words (categories='learned')
# Find words that occur after 'often' in the 'learned' category of the Brown corpus
words_after_often = sorted(set(b for (a, b) in nltk.bigrams (brown_learned_text) if a == 'often'))
print(words_after_often)
```

```
[',', '.', 'accomplished', 'analytically', 'appear', 'apt', 'associated', 'assuming', 'became', 'become', 'been', 'began', 'call', 'called', 'carefully', 'chose', 'classified
```

```
brown_lrnd_tagged = brown.tagged_words(categories='learned', tagset='universal')
tags = [b[1] for (a, b) in nltk.bigrams(brown_lrnd_tagged) if a[0] == 'often']
fd = nltk.FreqDist(tags)
fd.tabulate()
```

VERB	ADV	ADP	ADJ	.	PRT
37	8	7	6	4	2

```
from nltk.corpus import brown
import nltk
def process (sentence):
    for (w1, t1), (w2, t2), (w3, t3) in nltk.trigrams(sentence):
        if t1.startswith('V') and t2 == 'TO' and t3.startswith('V'):
            print(w1, w2, w3)
# Iterate through tagged sentences in the Brown corpus and apply the 'process' function
for tagged_sent in brown.tagged_sents():
    process (tagged_sent)
```

```
combined to achieve
continue to place
serve to protect
wanted to wait
allowed to place
expected to become
expected to approve
expected to make
intends to make
seek to set
like to see
designed to provide
get to hear
expects to tell
expected to give
prefer to pay
required to obtain
```

```
got to entertain
tried to keep
going to tell
thought to mix
going to jump
beginning to catch
try to see
phoned to say
bothering to look
forced to wipe
used to pretend
refused to approach
used to express
proceeds to lash
used to hang
seeks to expunge
trying to redeem
seemed to take
tried to conceal
came to know
refuses to continue
continue to scrape
given to understand
propose to vent
proceeded to mask
withhold to keep
begin to wither
help to intensify
seems to overtake
want to buy
```

```
from nltk.corpus import brown
import nltk
# Extract tagged words from the 'news' category of the Brown corpus using the Universal Tagset
brown_news_tagged = brown.tagged_words (categories='news', tagset='universal')
# Create a ConditionalFreqDist object based on the tagged words
data = nltk.ConditionalFreqDist((word.lower(), tag) for (word, tag) in brown_news_tagged)
# Iterate through words in sorted order and print words with more than three different tags
for word in sorted(data.conditions()):
    if len(data[word]) > 3:
        tags = [tag for (tag, _) in data[word].most_common()]
```

```
    print(word,''.join(tags))
```

```
best ADJADVVERBNOUN
close ADVADJVERBNOUN
open ADJVERBNOUNADV
present ADJADVNOUNVERB
that ADPDETPRONADV
```

```
pos = {} # Creating an empty dictionary named 'pos'
pos['colorless'] = 'ADJ' # Adding 'colorless' with the tag 'ADJ' to the dictionary
pos['ideas'] = 'N' # Adding 'ideas' with the tag 'N' to the dictionary
pos['sleep'] = 'V' # Adding 'sleep' with the tag 'V' to the dictionary
pos['furiously'] = 'ADV' # Adding 'furiously' with the tag 'ADV' to the dictionary
pos
```

```
{'colorless': 'ADJ', 'ideas': 'N', 'sleep': 'V', 'furiously': 'ADV'}
```

```
pos['ideas']
```

```
'N'
```

```
pos['colorless']
```

```
'ADJ'
```

```
list(pos) # [1]
sorted(pos) # [2]
[w for w in pos if w.endswith('s')]
```

```
['colorless', 'ideas']
```

```
for word in sorted(pos):
  print(word + ":", pos [word])
```

```
colorless: ADJ
furiously: ADV
ideas: N
sleep: V
```

```
pos = {
'colorless': 'ADJ',
'ideas': 'N',
'sleep': 'V',
'furiously': 'ADV'
}
# Retrieving and displaying keys, values, and items from the 'pos' dictionary
print(list(pos.keys())) # Retrieve keys
print(list(pos.values())) # Retrieve values
```

```
print(list(pos.items())) # Retrieve key-value pairs
# Iterating through sorted key-value pairs and printing them
for key, val in sorted(pos.items()):
   print (key + ":", val)
```

```
['colorless', 'ideas', 'sleep', 'furiously']
['ADJ', 'N', 'V', 'ADV']
[('colorless', 'ADJ'), ('ideas', 'N'), ('sleep', 'V'), ('furiously', 'ADV')]
colorless: ADJ
furiously: ADV
ideas: N
sleep: V
```

```
pos['sleep'] = 'V'
pos['sleep']
```

```
'V'
```

```
pos['sleep'] = 'N'
pos['sleep']
```

```
'N'
```

```
pos = {'colorless': 'ADJ', 'ideas': 'N', 'sleep': 'V', 'furiously': 'ADV'}
pos = dict(colorless='ADJ', ideas='N', sleep='V', furiously='ADV')
from collections import defaultdict
frequency = defaultdict(int)
frequency['colorless'] = 4
frequency['ideas'] # Accessing a key that doesn't exist defaults to 0
```

```
0
```

```
from collections import defaultdict
pos = defaultdict(list)
pos ['sleep'] = ['NOUN', 'VERB']
pos['ideas'] # Accessing a key that doesn't exist defaults to an empty list
# Output: [ ]
```

```
[]
```

```
from collections import defaultdict
pos = defaultdict(lambda: 'NOUN') # [1]
pos['colorless'] = 'ADJ'
pos['blog'] # [2]
# Output: 'NOUN'
```

```
'NOUN'
```

```
list(pos.items())
```

```
[('colorless', 'ADJ'), ('blog', 'NOUN')]
```

```
import nltk
from collections import defaultdict
nltk.download('gutenberg') # Download the gutenberg corpus
# Load the text and create a frequency distribution of words
alice = nltk.corpus.gutenberg.words('carroll-alice.txt')
vocab = nltk.FreqDist(alice)
# Select the 1000 most common words
v1000 = [word for (word, _) in vocab.most_common (1000)]
# Create a mapping dictionary to replace less common words with 'UNK'
mapping = defaultdict(lambda: 'UNK')
for v in v1000:
   mapping[v] = v
# Transform the original text, replacing less common words with 'UNK'
alice2 = [mapping[v] for v in alice]
# Display the initial 100 transformed words
print(alice2[:100])
# Determine the number of unique tokens after transformation
print(len(set(alice2)))
```

```
['[', 'Alice', "'", 's', 'Adventures', 'in', 'Wonderland', 'by', 'UNK', 'UNK', 'UNK', 'UNK', 'CHAPTER', 'I', '.', 'Down', 'the', 'Rabbit', '-', 'UNK', 'Alice', 'was', 'beginn
1001
[nltk_data] Downloading package gutenberg to /root/nltk_data...
[nltk_data]   Package gutenberg is already up-to-date!
```

```
from collections import defaultdict
from nltk.corpus import brown
from operator import itemgetter
counts = defaultdict(int)
for (word, tag) in brown.tagged_words(categories='news', tagset='universal'):
   counts [tag] += 1
counts ['NOUN']
sorted(counts)
sorted (counts.items(), key=itemgetter(1), reverse=True)
[t for t, c in sorted(counts.items(), key=itemgetter(1), reverse=True)]
```

```
['NOUN',
 'VERB',
 'ADP',
 '.',
 'DET',
 'ADJ',
 'ADV',
 'CONJ',
 'PRON',
 'PRT',
 'NUM',
 'X']
```

```
pair = ('NP', 8336)
pair[1]
itemgetter(1)(pair)
```

```
8336
```

```
from collections import defaultdict
import nltk
nltk.download('words') # Download the 'words' corpus
last_letters = defaultdict(list)
words = nltk.corpus.words.words('en')
for word in words:
    key = word[-2:]
    last_letters [key].append (word)
last_letters['ly']
last_letters['zy']
```

```
[nltk_data] Downloading package words to /root/nltk_data...
[nltk_data]   Package words is already up-to-date!
['blazy',
 'bleezy',
 'blowzy',
 'boozy',
 'breezy',
 'bronzy',
 'buzzy',
 'Chazy',
 'cozy',
 'crazy',
 'dazy',
 'dizzy',
 'dozy',
 'enfrenzy',
 'fezzy',
 'fizzy',
 'floozy',
 'fozy',
 'franzy',
 'frenzy',
 'friezy',
 'frizzy',
 'frowzy',
 'furzy',
 'fuzzy',
 'gauzy',
 'gazy',
 'glazy',
 'groszy',
 'hazy',
 'heezy',
 'Izzy',
 'jazzy',
 'Jozy',
 'lawzy',
 'lazy',
 'mazy',
 'mizzy',
 'muzzy',
 'nizy',
 'oozy',
 'quartzy',
 'quizzy',
 'twazzy',
 'unbreezy',
 'unfrizzy',
 'wheezy',
 'woozy',
 'wuzzy',
 'yezzy']
```

from collections import defaultdict

```
import nltk
anagrams = defaultdict(list)
words = nltk.corpus.words.words('en')
for word in words:
   key =''.join(sorted(word))
   anagrams [key].append (word)
anagrams ['aeilnrt']
```

```
['entrail', 'latrine', 'ratline', 'reliant', 'retinal', 'trenail']
```

```
import nltk
words = nltk.corpus.words.words('en')
anagrams = nltk.Index(((''.join(sorted(w)), w) for w in words)
anagrams ['aeilnrt']
```

```
['entrail', 'latrine', 'ratline', 'reliant', 'retinal', 'trenail']
```

```
from collections import defaultdict
import nltk
pos = defaultdict(lambda: defaultdict(int))
brown_news_tagged = brown.tagged_words (categories='news', tagset='universal')
for ((w1, t1), (w2, t2)) in nltk.bigrams (brown_news_tagged):
   pos[(t1, w2)][t2] += 1
pos [('DET', 'right')]
```

```
defaultdict(int, {'NOUN': 5, 'ADJ': 11})
```

```
from collections import defaultdict
import nltk
counts = defaultdict(int)
for word in nltk.corpus.gutenberg.words('milton-paradise.txt'):
   counts [word] += 1
[word for (word, count) in counts.items() if count == 32]
# Output: ['brought', 'Him', 'virtue', 'Against', 'There', 'thine', 'King', 'mortal', 'every', 'been']
```

```
['mortal',
 'Against',
 'Him',
 'There',
 'brought',
 'King',
 'virtue',
 'every',
 'been',
 'thine']
```

```
pos = {'colorless': 'ADJ', 'ideas': 'N', 'sleep': 'V', 'furiously': 'ADV'}
pos2 = dict((value, key) for (key, value) in pos.items())
pos2['N']
# Output: 'ideas'
```

```
'ideas'
```

```
from collections import defaultdict
# NLP_LAB5.ipynb - Colaboratory
pos = {'colorless': 'ADJ', 'ideas': 'N', 'sleep': 'V', 'furiously': 'ADV'}
pos.update({'cats': 'N', 'scratch': 'V', 'peacefully': 'ADV','old': 'ADJ'})
pos2 = defaultdict(list)
for key, value in pos.items():
   pos2[value].append(key)
pos2['ADV'];
import nltk
pos = {'colorless': 'ADJ', 'ideas': 'N', 'sleep': 'V', 'furiously': 'ADV'}
pos.update({'cats': 'N', 'scratch': 'V', 'peacefully': 'ADV', 'old': 'ADJ'})
pos2 = nltk.Index((value, key) for (key, value) in pos.items())
pos2['ADV']
# Output: ['peacefully', 'furiously']
```

```
['furiously', 'peacefully']
```

```
from nltk.corpus import brown
#Fetching tagged sentences from the 'news' category in the Brown corpus
brown_tagged_sents = brown.tagged_sents (categories='news')
# Fetching untagged sentences from the 'news' category in the Brown corpus
brown_sents = brown.sents(categories='news')
tags = [tag for (word, tag) in brown.tagged_words (categories='news')]
nltk.FreqDist (tags).max()
```

```
'NN'
```

```
raw = 'I do not like green eggs and ham, I do not like them Sam I am!'
tokens = nltk.word_tokenize(raw)
default_tagger = nltk.DefaultTagger('NN')
default_tagger.tag (tokens)
```

```
[('I', 'NN'),
 ('do', 'NN'),
 ('not', 'NN'),
 ('like', 'NN'),
 ('green', 'NN'),
 ('eggs', 'NN'),
 ('and', 'NN'),
 ('ham', 'NN'),
 (',', 'NN'),
 ('I', 'NN'),
 ('do', 'NN'),
 ('not', 'NN'),
 ('like', 'NN'),
 ('them', 'NN'),
 ('Sam', 'NN'),
 ('I', 'NN'),
 ('am', 'NN'),
 ('!', 'NN')]
```

```
default_tagger.evaluate(brown_tagged_sents)
```

```
/tmp/ipykernel_7143/735304995.py:1: DeprecationWarning:
  Function evaluate() has been deprecated.  Use accuracy(gold)
  instead.
  default_tagger.evaluate(brown_tagged_sents)
0.13089484257215028
```

```
patterns = [
(r'.*ing$', 'VBG'), # gerunds
# simple past
# 3rd singular present
# modals
(r'.*ed$', 'VBD'),
(r'.*es$', 'VBZ'),
(r'.*ould$', 'MD'),
(r'.*\'s$', 'NN$'),
(r'.*s$', 'NNS'),
(r'^-?[0-9]+(\.[0-9]+)?$', 'CD'), # cardinal numbers
(r'.*', 'NN') # nouns (default)
```

```
# possessive nouns
# plural nouns
]
regexp_tagger = nltk.RegexpTagger (patterns)
regexp_tagger.tag(brown_sents [3])
```

```
[(' ', 'NN'),
 ('Only', 'NN'),
 ('a', 'NN'),
 ('relative', 'NN'),
 ('handful', 'NN'),
 ('of', 'NN'),
 ('such', 'NN'),
 ('reports', 'NNS'),
 ('was', 'NNS'),
 ('received', 'VBD'),
 ("''", 'NN'),
 (',', 'NN'),
 ('the', 'NN'),
 ('jury', 'NN'),
 ('said', 'NN'),
 (',', 'NN'),
 ('``', 'NN'),
 ('considering', 'VBG'),
 ('the', 'NN'),
 ('widespread', 'NN'),
 ('interest', 'NN'),
 ('in', 'NN'),
 ('the', 'NN'),
 ('election', 'NN'),
 (',', 'NN'),
 ('the', 'NN'),
 ('number', 'NN'),
 ('of', 'NN'),
 ('voters', 'NNS'),
 ('election', 'NN'),
 (',', 'NN'),
 ('the', 'NN'),
 ('number', 'NN'),
 ('of', 'NN'),
 ('voters', 'NNS'),
 ('and', 'NN'),
 ('the', 'NN'),
 ('size', 'NN'),
 ('of', 'NN'),
 ('this', 'NNS'),
 ('city', 'NN'),
 ("''", 'NN'),
 ('.', 'NN')]
```

```
regexp_tagger.evaluate (brown_tagged_sents)
```

```
/tmp/ipykernel_7143/1522720769.py:1: DeprecationWarning:
  Function evaluate() has been deprecated.  Use accuracy(gold)
  instead.
  regexp_tagger.evaluate (brown_tagged_sents)
0.20186168625812995
```

```
import nltk
fd = nltk.FreqDist(brown.words (categories='news'))
cfd = nltk.ConditionalFreqDist(brown.tagged_words (categories='news'))
most_freq_words = fd.most_common (100)
likely_tags = dict((word, cfd[word].max()) for (word, _) in most_freq_words)
baseline_tagger = nltk.UnigramTagger (model=likely_tags)
baseline_tagger.evaluate(brown_tagged_sents)
```

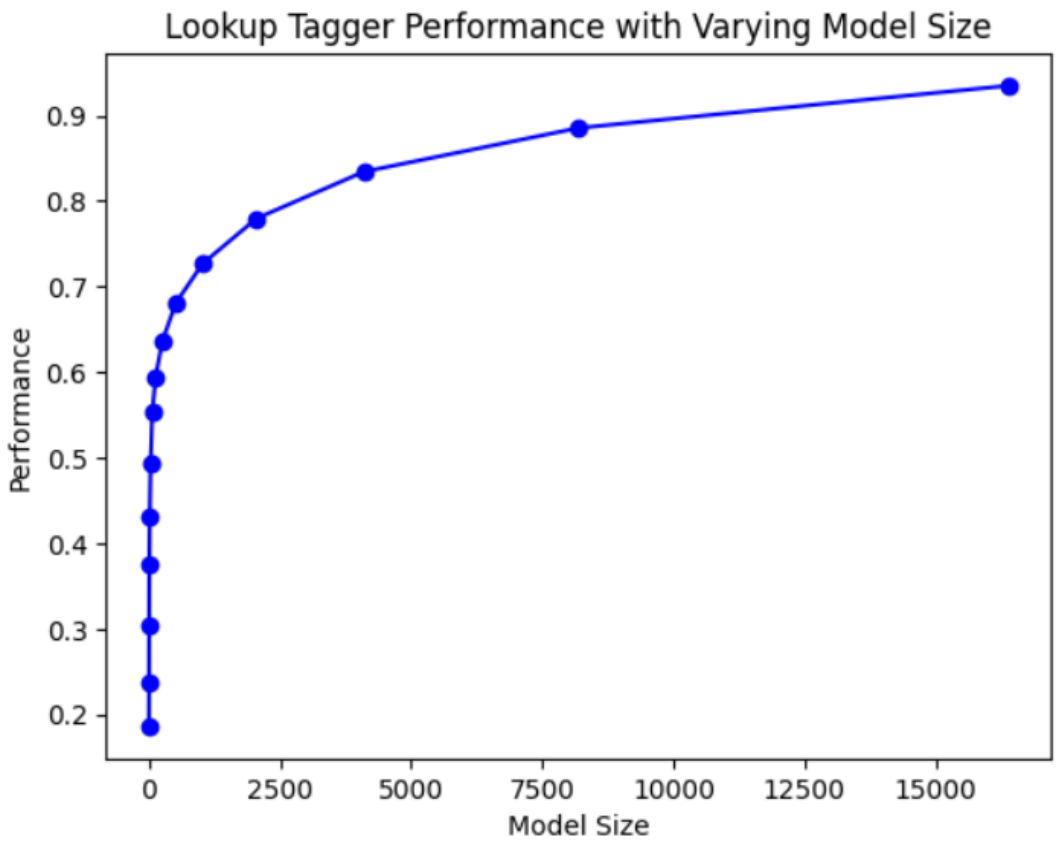
```
/tmp/ipykernel_7143/3538440167.py:7: DeprecationWarning:
  Function evaluate() has been deprecated.  Use accuracy(gold)
  instead.
  baseline_tagger.evaluate(brown_tagged_sents)
0.45578495136941344
```

```
import nltk
sent = brown.sents (categories='news') [3]
baseline_tagger.tag(sent)
```

```
[('``', '``'),
 ('Only', None),
 ('a', 'AT'),
 ('relative', None),
 ('handful', None),
 ('of', 'IN'),
 ('such', None),
 ('reports', None),
 ('was', 'BEDZ'),
 ('received', None),
 ("''", "''"),
 (',', ','),
 ('the', 'AT'),                    ('voters', None),
 ('jury', None),                   ('and', 'CC'),
 ('said', 'VBD'),                  ('the', 'AT'),
 (',', ','),                       ('size', None),
 ('``', '``'),                     ('of', 'IN'),
 ('considering', None),            ('this', 'DT'),
 ('the', 'AT'),                    ('city', None),
 ('widespread', None),             ("''", "''"),
 ('interest', None),               ('.', '.')]
```

```
import nltk
baseline_tagger = nltk.UnigramTagger (model=likely_tags, backoff=nltk.DefaultTagger('NN'))
def performance(cfd, wordlist):
   lt = dict((word, cfd[word].max()) for word in wordlist)
   baseline_tagger = nltk.UnigramTagger (model=lt, backoff=nltk.DefaultTagger('NN'))
```

```
    return baseline_tagger.accuracy(brown.tagged_sents (categories='news')) # Changed evaluate() to
accuracy()
def display():
    import pylab
    word_freqs = nltk.FreqDist(brown.words (categories='news')).most_common()
    words_by_freq = [w for (w, _) in word_freqs]
    cfd = nltk.ConditionalFreqDist(brown.tagged_words (categories='news'))
    sizes = 2 ** pylab.arange(15)
    perfs = [performance(cfd, words_by_freq[:size]) for size in sizes]
    pylab.plot(sizes, perfs, '-bo')
    pylab.title('Lookup Tagger Performance with Varying Model Size')
    pylab.xlabel('Model Size')
    pylab.ylabel('Performance')
    pylab.show()
display()
```




```
from nltk.corpus import brown
brown_tagged_sents = brown.tagged_sents (categories='news')
brown_sents = brown.sents(categories='news')
unigram_tagger = nltk.UnigramTagger (brown_tagged_sents)
unigram_tagger.tag(brown_sents [2007])
```

```
[('Various', 'JJ'),
 ('of', 'IN'),
 ('the', 'AT'),
 ('apartments', 'NNS'),
 ('are', 'BER'),
 ('of', 'IN'),
 ('the', 'AT'),
 ('terrace', 'NN'),
 ('type', 'NN'),
 (',', ','),
 ('being', 'BEG'),
 ('on', 'IN'),
 ('the', 'AT'),
 ('ground', 'NN'),
 ('floor', 'NN'),
 ('so', 'QL'),
 ('that', 'CS'),
 ('entrance', 'NN'),
 ('is', 'BEZ'),
 ('direct', 'JJ'),
 ('.', '.')]
```

```
unigram_tagger.evaluate(brown_tagged_sents)
```

```
/tmp/ipykernel_7143/1449911821.py:1: DeprecationWarning:
  Function evaluate() has been deprecated.  Use accuracy(gold)
  instead.
  unigram_tagger.evaluate(brown_tagged_sents)
0.9349006503968017
```

```
size = int(len(brown_tagged_sents) * 0.9)
train_sents = brown_tagged_sents[:size]
test_sents = brown_tagged_sents[size:]
unigram_tagger = nltk.UnigramTagger(train_sents)
unigram_tagger.evaluate(test_sents)
```

```
/tmp/ipykernel_7143/3093492045.py:5: DeprecationWarning:
  Function evaluate() has been deprecated.  Use accuracy(gold)
  instead.
  unigram_tagger.evaluate(test_sents)
0.8121200039868434
```

```
bigram_tagger = nltk.BigramTagger(train_sents)
bigram_tagger.tag(brown_sents [2007])
```

```
[( various ,  JJ ),
 ('of', 'IN'),
 ('the', 'AT'),
 ('apartments', 'NNS'),
 ('are', 'BER'),
 ('of', 'IN'),
 ('the', 'AT'),
 ('terrace', 'NN'),
 ('type', 'NN'),
 (',', ','),
 ('being', 'BEG'),
 ('on', 'IN'),
 ('the', 'AT'),
 ('ground', 'NN'),
 ('floor', 'NN'),
 ('so', 'CS'),
 ('that', 'CS'),
 ('entrance', 'NN'),
 ('is', 'BEZ'),
 ('direct', 'JJ'),
 ('.', '.')]
```

unseen_sent = brown_sents [4203]
bigram_tagger.tag(unseen_sent)

```
[('The', 'AT'),
 ('population', 'NN'),
 ('of', 'IN'),
 ('the', 'AT'),
 ('Congo', 'NP'),
 ('is', 'BEZ'),
 ('13.5', None),
 ('million', None),
 (',', None),
 ('divided', None),
 ('into', None),
 ('at', None),
 ('least', None),
 ('seven', None),
 ('major', None),
 ('``', None),
 ('culture', None),
 ('clusters', None),
 ("''", None),
 ('and', None),
 ('innumerable', None),
 ('tribes', None),
 ('speaking', None),
 ('400', None),
 ('separate', None),
 ('dialects', None),
 ('.', None)]
```

bigram_tagger.evaluate(test_sents)

```
/tmp/ipykernel_7143/63762215.py:1: DeprecationWarning:
  Function evaluate() has been deprecated.  Use accuracy(gold)
  instead.
  bigram_tagger.evaluate(test_sents)
0.10206319146815508
```

t0 = nltk.DefaultTagger('NN')
t1 = nltk.UnigramTagger(train_sents, backoff=t0)
t2 = nltk.BigramTagger(train_sents, backoff=t1)
t2.evaluate(test_sents)

```
/tmp/ipykernel_7143/2463049084.py:4: DeprecationWarning:
  Function evaluate() has been deprecated.  Use accuracy(gold)
  instead.
  t2.evaluate(test_sents)
0.8452108043456593
```

```
from pickle import dump
output = open('t2.pkl', 'wb')
dump(t2, output, -1)
output.close()
from pickle import load
input_file = open('t2.pkl', 'rb')
tagger = load(input_file)
input_file.close()
text = """The board's action shows what free enterprise
is up against in our complex maze of regulatory laws."""
tokens = text.split()
tagger.tag(tokens)
```

```
[('The', 'AT'),
 ("board's", 'NN$'),
 ('action', 'NN'),
 ('shows', 'NNS'),
 ('what', 'WDT'),
 ('free', 'JJ'),
 ('enterprise', 'NN'),
 ('is', 'BEZ'),
 ('up', 'RP'),
 ('against', 'IN'),
 ('in', 'IN'),
 ('our', 'PP$'),
 ('complex', 'JJ'),
 ('maze', 'NN'),
 ('of', 'IN'),
 ('regulatory', 'NN'),
 ('laws.', 'NN')]
```

```
cfd = nltk.ConditionalFreqDist(
((x[1], y[1], z[0]), z[1])
for sent in brown_tagged_sents
for x, y, z in nltk.trigrams(sent))
ambiguous_contexts = [c for c in cfd.conditions() if len(cfd[c]) > 1]
sum(cfd[c].N() for c in ambiguous_contexts) / cfd.N()
```

```
0.049297702068029296
```

```
test_tags = [tag for sent in brown.sents(categories='editorial')
for (word, tag) in t2.tag(sent)]
```

```
gold_tags = [tag for (word, tag) in brown.tagged_words(categories='editorial')]
print(nltk.ConfusionMatrix(gold_tags, test_tags))
```

```
  FW-JJ-TL |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
     FW-NN |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
  FW-NN-TL |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
    FW-NNS |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
     FW-NP |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
    FW-PPL |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
    FW-PPO |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
     FW-VB |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
    FW-VBZ |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
    FW-WDT |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
        HV |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
       HVD |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
       HVG |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
       HVN |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
       HVZ |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
      HVZ* |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
        IN |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    3    .    .
     IN-HL |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
     IN-NC |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
     IN-TL |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
        JJ |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    8    .    .
     JJ-HL |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    1    .    .
     JJ-TL |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
  JJ-TL-HL |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
       JJR |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    2    .    .
    JJR-HL |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
    JJR-TL |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
       JJS |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .

       VBN |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
    VBN-HL |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
    VBN-TL |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
 VBN-TL-HL |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
       VBZ |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
    VBZ-HL |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
    VBZ-TL |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
       WDT |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
   WDT+BEZ |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
WDT+BEZ-HL |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
    WDT-HL |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
       WP$ |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
       WPO |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
    WPO-TL |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
       WPS |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
       WRB |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
    WRB-HL |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
        `` |    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .    .
-----------+-------------------------------------------------------------------------------------------------------------------------------
(row = reference; col = test)
```

```
from nltk.tbl import demo as brill_demo
brill_demo.demo()
```

```
Loading tagged data from treebank...
Read testing data (200 sents/5251 wds)
Read training data (800 sents/19933 wds)
Read baseline data (800 sents/19933 wds) [reused the training set]
Trained baseline tagger
    Accuracy on test set: 0.8358
```

EXPERIMENT-6
HMM(Hidden Markov Model) - POS Tagging Algorithm

```
import nltk, re, pprint
import numpy as np
import pandas as pd
import requests
import matplotlib.pyplot as plt
import seaborn as sns
import pprint, time
import random
from sklearn.model_selection import train_test_split
from nltk.tokenize import word_tokenize

nltk.download('treebank') # Download the 'treebank' corpus
nltk.download('universal_tagset') # Download the 'universal_tagset' resource

# reading the Treebank tagged sentences
wsj = list(nltk.corpus.treebank.tagged_sents(tagset='universal'))
```

```
[nltk_data] Downloading package treebank to /root/nltk_data...
[nltk_data]   Unzipping corpora/treebank.zip.
[nltk_data] Downloading package universal_tagset to /root/nltk_data...
[nltk_data]   Unzipping taggers/universal_tagset.zip.
```

```
print(wsj[:40])
```

```
[[('Pierre', 'NOUN'), ('Vinken', 'NOUN'), (',', '.'), ('61', 'NUM'), ('years', 'NOUN'), ('old', 'ADJ'), (',', '.'), ('will', 'VERB'), ('join', 'VERB'), ('the', 'DET'), ('boa
```

```
random.seed (1234)
train_set, test_set = train_test_split(wsj,test_size=0.05)
print(len(train_set))
print(len(test_set))
print(train_set[:40])
```

```
3718
196
[[('Only', 'ADV'), ('19', 'NUM'), ('%', 'NOUN'), ('of', 'ADP'), ('the', 'DET'), ('purchasing', 'VERB'), ('managers', 'NOUN'), ('reported', 'VERB'), ('better', 'ADJ'),
```

```
train_tagged_words = [tup for sent in train_set for tup in sent]
len(train_tagged_words)
```

```
95598
```

```
tokens = [pair[0] for pair in train_tagged_words]
tokens [:10]
```

```
['Only',
 '19',
 '%',
 'of',
 'the',
 'purchasing',
 'managers',
 'reported',
 'better',
 'export']
```

```
V = set(tokens)
print(len(V))
```

```
12080
```

```
T = set([pair[1] for pair in train_tagged_words])
len(T)
```

```
12
```

```
print(T)
```

```
{'ADV', 'DET', 'NUM', 'X', 'VERB', 'NOUN', 'PRT', 'PRON', 'CONJ', 'ADP', 'ADJ', '.'}
```

HMM(Hidden Markov Model) - POS Tagging Algorithm Emission Probabilities

```
t = len(T)
v = len(V)
w_given_t = np.zeros((t, v))
# compute word given tag: Emission Probability
def word_given_tag(word, tag, train_bag = train_tagged_words):
    tag_list = [pair for pair in train_bag if pair[1]==tag]
    count_tag = len(tag_list)
    w_given_tag_list = [pair[0] for pair in tag_list if pair[0]==word]
    count_w_given_tag = len(w_given_tag_list)
    return (count_w_given_tag, count_tag)
# examples
# large
print("\n", "large")
print(word_given_tag('large', 'ADJ'))
print(word_given_tag('large', 'VERB'))
print(word_given_tag('large', 'NOUN'), "\n")
# will
print("\n", "will")
print(word_given_tag('will', 'MD'))
print(word_given_tag('will', 'NOUN'))
print(word_given_tag('will', 'VERB'))
# book
```

```
print("\n", "book")
print(word_given_tag('book', 'NOUN'))
print(word_given_tag('book', 'VERB'))
```

```
 large
(26, 6105)
(0, 12870)
(0, 27469)


 will
(0, 0)
(1, 27469)
(262, 12870)

 book
(6, 27469)
(1, 12870)
```

```
word_given_tag('Twitter', 'NOUN')
```

```
(0, 27469)
```

Transisition Probabilities

```
def t2_given_t1(t2, t1, train_bag = train_tagged_words):
    tags = [pair[1] for pair in train_bag]
    count_t1 = len([t for t in tags if t==t1])
    count_t2_t1 = 0
    for index in range(len(tags)-1):
        if tags[index]==t1 and tags[index+1] == t2:
            count_t2_t1 += 1
    return (count_t2_t1, count_t1)

# examples
print(t2_given_t1(t2='NOUN', t1='ADJ'))
print(t2_given_t1('ADJ', 'NOUN'))
print(t2_given_t1('NOUN', 'DET'))
print(t2_given_t1('NOUN', 'VERB'))
print(t2_given_t1(',', 'NOUN'))
print(t2_given_t1('PRT', 'PRT'))
print(t2_given_t1('VERB', 'NOUN'))
```

```
(4285, 6105)
(335, 27469)
(5279, 8280)
(1431, 12870)
(0, 27469)
(6, 3042)
(4042, 27469)
```

```
#Please note P(tag|start) is same as P(tag|'.')
print(t2_given_t1('DET', '.'))
print(t2_given_t1('VERB', '.'))
print(t2_given_t1('NOUN', '.'))
print(t2_given_t1('NOUN', '.'))
```

```
(1939, 11108)
(989, 11108)
(2465, 11108)
(2465, 11108)
```

```
t2_given_t1('VERB', 'DET')
```

```
(328, 8280)
```

```
# creating txt transition matrix of tags
# each column is t2, each row is t1
# thus M(i, j) represents P(tj given ti)
tags_matrix = np.zeros((len(T), len(T)), dtype='float32')
for i, t1 in enumerate(list(T)):
    for j, t2 in enumerate(list(T)):
        tags_matrix[i, j] = t2_given_t1(t2, t1)[0]/t2_given_t1(t2, t1)[1]
tags_matrix
```

```
array([[7.67441839e-02, 6.77740872e-02, 3.08970101e-02, 2.35880390e-02,
        3.48172754e-01, 3.22259143e-02, 1.39534883e-02, 1.52823916e-02,
        6.97674416e-03, 1.15946844e-01, 1.29235879e-01, 1.39202654e-01],
       [1.24396132e-02, 5.55555569e-03, 2.22222228e-02, 4.60144915e-02,
        3.96135263e-02, 6.37560368e-01, 2.41545888e-04, 3.50241549e-03,
        4.83091775e-04, 9.05797072e-03, 2.05314010e-01, 1.79951694e-02],
       [2.98151467e-03, 3.27966618e-03, 1.85748354e-01, 2.08109722e-01,
        1.78890880e-02, 3.55396539e-01, 2.71317828e-02, 1.49075733e-03,
        1.37149673e-02, 3.33929621e-02, 3.30948122e-02, 1.17769830e-01],
       [2.57311817e-02, 5.48186041e-02, 2.71695689e-03, 7.49560520e-02,
        2.06169084e-01, 6.16909079e-02, 1.82835221e-01, 5.54578863e-02,
        1.03883650e-02, 1.46875501e-01, 1.69410259e-02, 1.61419213e-01],
       [8.16627815e-02, 1.34343430e-01, 2.24553216e-02, 2.17715621e-01,
        1.68142974e-01, 1.11188814e-01, 3.13131325e-02, 3.59751359e-02,
        5.43900533e-03, 9.09090936e-02, 6.59673661e-02, 3.48873362e-02],
       [1.71830058e-02, 1.32877063e-02, 9.61083360e-03, 2.85412651e-02,
        1.47147700e-01, 2.64407158e-01, 4.39404435e-02, 4.84182173e-03,
        4.24114466e-02, 1.76671878e-01, 1.21955657e-02, 2.39761189e-01],
       [1.01906639e-02, 1.00262985e-01, 5.71992099e-02, 1.34779746e-02,
        3.97435904e-01, 2.48191983e-01, 1.97238661e-03, 1.80802103e-02,
        1.97238661e-03, 2.13675220e-02, 8.64562765e-02, 4.33925055e-02],
       [3.42571214e-02, 1.00076981e-02, 7.31331808e-03, 9.43033099e-02,
        4.81139332e-01, 2.09776744e-01, 1.23171667e-02, 7.31331808e-03,
        5.38876047e-03, 2.23248657e-02, 7.54426494e-02, 4.04157043e-02],
       [5.45200370e-02, 1.18359737e-01, 4.14725058e-02, 6.98974822e-03,
        1.57036349e-01, 3.50419372e-01, 4.19384893e-03, 6.01118356e-02,
        4.65983234e-04, 5.35880700e-02, 1.18825719e-01, 3.40167768e-02],
       [1.38903726e-02, 3.23645681e-01, 6.23998307e-02, 3.47259305e-02,
        8.54792167e-03, 3.22577208e-01, 1.28218823e-03, 6.93450123e-02,
        8.54792190e-04, 1.66684482e-02, 1.06528476e-01, 3.95341367e-02],
       [4.75020474e-03, 4.75020474e-03, 1.98198203e-02, 2.01474205e-02,
        1.19574117e-02, 7.01883674e-01, 1.09746112e-02, 4.91400482e-04,
        1.67076159e-02, 7.81326815e-02, 6.63390681e-02, 6.40458614e-02],
       [5.26647456e-02, 1.74558878e-01, 8.07526112e-02, 2.72776373e-02,
        8.90349299e-02, 2.21912131e-01, 2.43068067e-03, 6.48181513e-02,
        5.79762347e-02, 9.09254625e-02, 4.49225791e-02, 9.26359370e-02]],
      dtype=float32)
```

```
# convert the matrix to a df for better readability
tags_df = pd.DataFrame(tags_matrix, columns = list(T), index=list(T))
tags_df
```

	ADV	DET	NUM	X	VERB	NOUN	PRT	PRON	CONJ	ADP	ADJ	.
ADV	0.076744	0.067774	0.030897	0.023588	0.348173	0.032226	0.013953	0.015282	0.006977	0.115947	0.129236	0.139203
DET	0.012440	0.005556	0.022222	0.046014	0.039614	0.637560	0.000242	0.003502	0.000483	0.009058	0.205314	0.017995
NUM	0.002982	0.003280	0.185748	0.208110	0.017889	0.355397	0.027132	0.001491	0.013715	0.033393	0.033095	0.117770
X	0.025731	0.054819	0.002717	0.074956	0.206169	0.061691	0.182835	0.055458	0.010388	0.146876	0.016941	0.161419
VERB	0.081663	0.134343	0.022455	0.217716	0.168143	0.111189	0.031313	0.035975	0.005439	0.090909	0.065967	0.034887
NOUN	0.017183	0.013288	0.009611	0.028541	0.147148	0.264407	0.043940	0.004842	0.042411	0.176672	0.012196	0.239761
PRT	0.010191	0.100263	0.057199	0.013478	0.397436	0.248192	0.001972	0.018080	0.001972	0.021368	0.086456	0.043393
PRON	0.034257	0.010008	0.007313	0.094303	0.481139	0.209777	0.012317	0.007313	0.005389	0.022325	0.075443	0.040416
CONJ	0.054520	0.118360	0.041473	0.006990	0.157036	0.350419	0.004194	0.060112	0.000466	0.053588	0.118826	0.034017
ADP	0.013890	0.323646	0.062400	0.034726	0.008548	0.322577	0.001282	0.069345	0.000855	0.016668	0.106528	0.039534
ADJ	0.004750	0.004750	0.019820	0.020147	0.011957	0.701884	0.010975	0.000491	0.016708	0.078133	0.066339	0.064046
.	0.052665	0.174559	0.080753	0.027278	0.089035	0.221912	0.002431	0.064818	0.057976	0.090925	0.044923	0.092636

tags_df.loc['.', :]

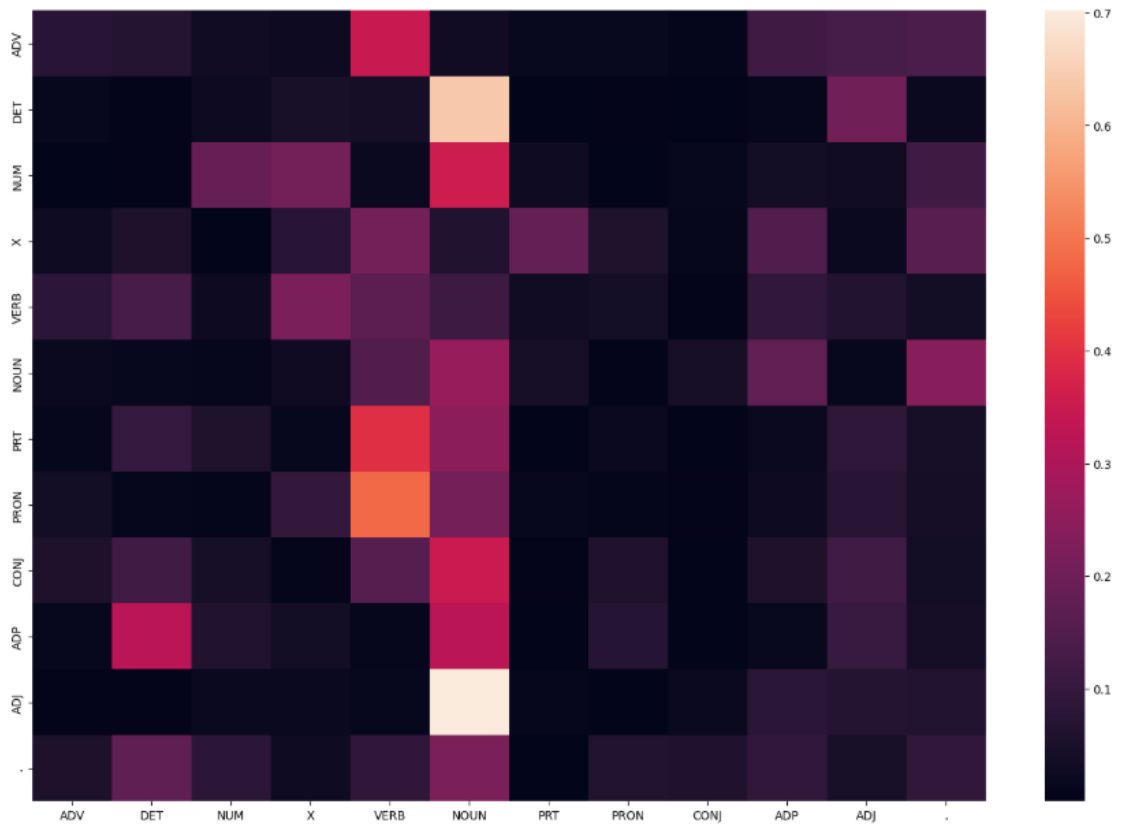
	.
ADV	0.052665
DET	0.174559
NUM	0.080753
X	0.027278
VERB	0.089035
NOUN	0.221912
PRT	0.002431
PRON	0.064818
CONJ	0.057976
ADP	0.090925
ADJ	0.044923
.	0.092636

dtype: float32

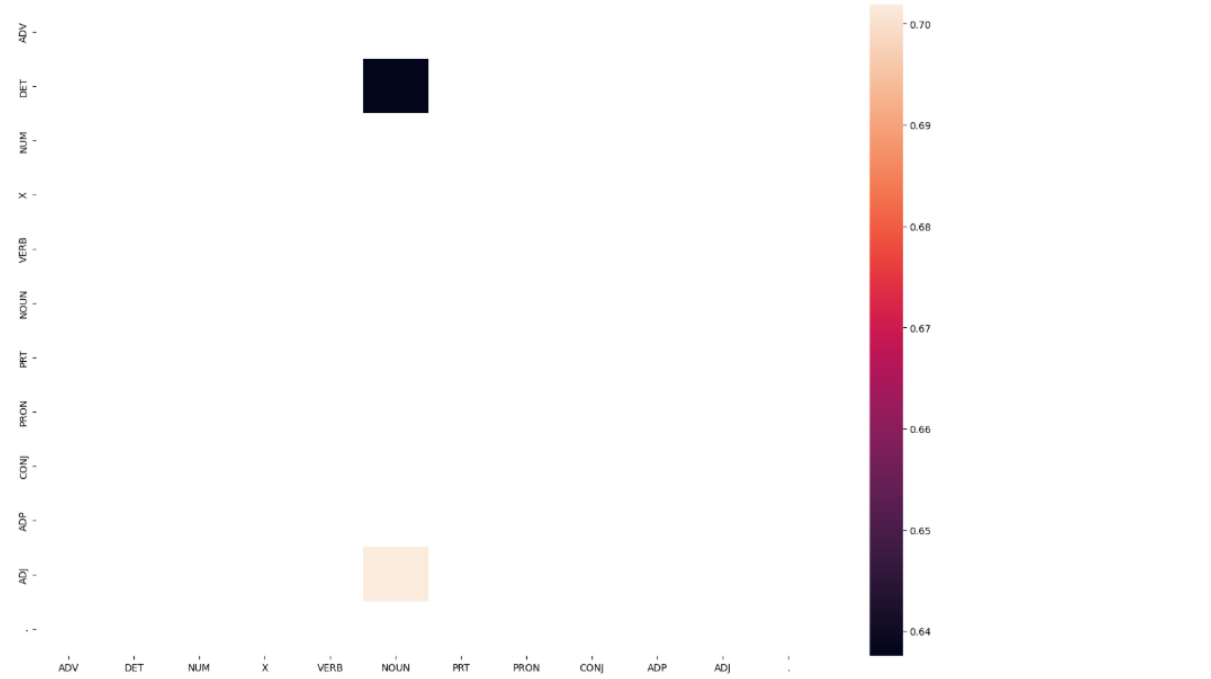
Heat Map of Tags Matrix

T(i, j) means P(tag j given tag i)

```
plt.figure(figsize=(18, 12))
sns.heatmap(tags_df)
plt.show()
```




```
# frequent tags
# filter the df to get P(t2, t1) > 0.5
tags_frequent = tags_df[tags_df>0.5]
plt.figure(figsize=(18, 12))
sns.heatmap(tags_frequent)
plt.show()
```



Vanilla Viterbi

```
def Viterbi_vanilla(words, train_bag = train_tagged_words):
    state = []
    T = list(set([pair[1] for pair in train_bag]))
    for key, word in enumerate(words):
        #initialise list of probability column for a given observation
        p = []
        for tag in T:
            if key == 0:
                transition_p = tags_df.loc['.', tag]
            else:
                transition_p = tags_df.loc[state[-1], tag]
            # compute emission and state probabilities
            emission_p = word_given_tag(words [key], tag)[0]/word_given_tag(words[key], tag) [1]
            state_probability = emission_p * transition_p
            p.append(state_probability)
        #print('word is {} and list is {}'.format(word,p))
        pmax = max(p)
        # getting state for which probability is maximum
        state_max = T[p.index(pmax)]
        state.append(state_max)
    return list(zip(words, state))
```

Vanilla Laplace smoothened

```
def Viterbi_smoothed(words, train_bag = train_tagged_words):
    state = []
```

```
    T = list(set([pair[1] for pair in train_bag]))
    for key, word in enumerate(words):
        #initialise list of probability column for a given observation
        p = []
        for tag in T:
            if key == 0:
                transition_p = tags_df.loc['.', tag]
            else:
                transition_p = tags_df.loc[state[-1], tag]
            # compute emission and state probabilities
            emission_p = 0.000001+word_given_tag(words [key], tag)[0]/word_given_tag(words [key], tag)
[1]+0.000001*len(T)
            state_probability = emission_p * transition_p
            p.append(state_probability)
        #print('word is {} and list is {}'.format(word,p))
        pmax = max(p)
        # getting state for which probability is maximum
        state_max = T[p.index(pmax)]
        state.append(state_max)
    return list(zip(words, state))
```

Viterbi with RegExp for unknown words

```
def find_state(word):
    #punc=['"',"'",',','.','(',')','?','[',']',':',';']
    # xc=['*'] # This was likely intended for the X tag logic below, but not needed as a separate list.
    if re.search(r'.*(ing|ed|es|ould)$', word.lower()):
        return 'VERB'
    elif re.search(r'to$', str(word).lower()):
        return 'PRT'
    elif re.search(r'^-?[0-9]+(\.[0-9]+)?$', str(word).lower()):
        return 'NUM'
    elif '*' in word:
        return 'X'
    elif re.search(r'.*\'s$', word.lower()):
        return 'NOUN'
    elif re.search(r'.*ness$', word.lower()):
        return 'NOUN'
    elif re.search(r'(The|the|A|a|An|an)$', word):
        return 'DET'
    elif re.search(r'.*able$', word.lower()):
        return 'ADJ'
    elif re.search(r'.*ly$', word.lower()):
        return 'ADV'
    # NLP_LAB6.ipynb - Colaboratory
```

```
    elif re.search(r'(He|he|She|she|It|it|I|me|Me|You|you|His|his|Her|her|Its|its|my|Your
|your|Yours|yours)$', word):
        return 'PRON'
    elif
re.search(r'(on|On|at|At|since|Since|For|for|Ago|ago|before|Before|till|Till|until|Until|by|By
|Beside|beside|under|Under|below|Below)', word):
        return 'ADP'
    #elif re.search(r'', word): # This condition is always true and should be removed or made more
specific
    #    return 'NOUN'
    #elif re.search(r'(\']"]\.[\S\[\[\?[\S\[\[\][\:[\;)+', word): # Invalid regex, commenting out
    #    pass
    else:
        # Default tag, if no other rule matches
        return 'NOUN'


def Viterbi_manual(words, train_bag = train_tagged_words):
    state = []
    T = list(set([pair[1] for pair in train_bag]))
    for key, word in enumerate(words):
        #initialise list of probability column for a given observation
        p= []
        for tag in T:
            if key == 0:
                transition_p = tags_df.loc['.', tag]
            else:
                transition_p = tags_df.loc[state[-1], tag]
            # compute emission and state probabilities
            emission_p = word_given_tag(words[key], tag)[0]/word_given_tag(words [key], tag) [1]
            state_probability = emission_p * transition_p
            p.append(state_probability)
        #print('word is {} and list is {}'.format(word,p))
        pmax = max(p)
        if pmax == 0.0:
            state_word = find_state(word)
            state.append(state_word)
        else:
            # getting state for which probability is maximum
            state_max = T[p.index(pmax)]
            state.append(state_max)
    return list(zip(words, state))
```

Viterbi for unknown words given mostly used tag

```
def Viterbi_common(words, train_bag = train_tagged_words):
    state = []
    T = list(set([pair[1] for pair in train_bag]))
    for key, word in enumerate(words):
        #initialise list of probability column for a given observation
        p = []
        for tag in T:
            if key == 0:
                # This part is missing a calculation for the initial probability
                # For now, let's assume a default small value or handle it later
                transition_p = tags_df.loc['.', tag]
            else:
                transition_p = tags_df.loc[state[-1], tag]
            # compute emission and state probabilities
            if word in tokens:
                emission_p = word_given_tag(words[key], tag)[0]/word_given_tag(words[key], tag)[1]
            else:
                emission_p=1 # Assign a default emission probability for unknown words
            state_probability = emission_p * transition_p
            p.append(state_probability)
        #print('word is {} and list is {}'.format(word,p))
        pmax = max(p)
        if pmax==0.0:
            # If no probability found, assign a common tag, e.g., 'NOUN'
            state_word='NOUN'
            state.append(state_word)
        else:
            #print(word,':', state_word)
            # getting state for which probability is maximum
            state_max = T[p.index(pmax)]
            state.append(state_max)
    return list(zip(words, state))


def Viterbi(words, train_bag = train_tagged_words):
    state = []
    T = list(set([pair[1] for pair in train_bag]))
    for key, word in enumerate(words):
        #initialise list of probability column for a given observation
        p = []
        for tag in T:
            if key == 0:
                transition_p = tags_df.loc['.', tag]
            else:
                transition_p = tags_df.loc[state[-1], tag]
            # compute emission and state probabilities
```

```
        if word in tokens:
            emission_p = word_given_tag(words[key], tag)[0]/word_given_tag(words [key], tag) [1]
        else:
            emission_p=1
        state_probability = emission_p * transition_p
        p.append(state_probability)
    #print('word is {} and list is {}'.format(word,p))
    pmax = max(p)
    # getting state for which probability is maximum
    state_max = T[p.index(pmax)]
    state.append(state_max)
  return list(zip(words, state))
```

Testing our model on Vanilla Viterbi

```
import time

# tagging the test sentences
start = time.time ()
tagged_seq_vanilla = Viterbi_vanilla(test_tagged_words)
check = [i for i, j in zip(tagged_seq_vanilla, test_run_base) if i == j]
accuracy = len(check)/len(tagged_seq_vanilla)
end = time.time()
print('Time Taken:',end-start)
print('Accuracy is :',accuracy)
```

```
Time Taken: 709.7555644512177
Accuracy is : 0.9052776683733753
```

Testing model on Viterbi Laplace smoothened

```
start = time.time()
tagged_seq_smooth = Viterbi_smoothed(test_tagged_words)
check = [i for i, j in zip(tagged_seq_smooth, test_run_base) if i == j]
accuracy = len(check)/len(tagged_seq_smooth)
end = time.time()
print('Time Taken:',end-start)
print('Accuracy is :',accuracy)
print('Laplace smoothed Viterbi-Incorrect tagged cases are:')
incorrect_tagged_cases = [[test_run_base[i-1],j] for i, j in enumerate(zip(tagged_seq_smooth,
test_run_base)) if j[0]!=j[1]]
[i[1]for i in incorrect_tagged_cases]
```

```
Time Taken: 722.4721169471741
Accuracy is : 0.935998424576605
Laplace smoothed Viterbi-Incorrect tagged cases are:
[(('2.75', 'NOUN'), ('2.75', 'NUM')),
 (('less', 'ADJ'), ('less', 'ADV')),
 (('tunes', 'X'), ('tunes', 'NOUN')),
 (('continental', 'DET'), ('continental', 'ADJ')),
 (('English', 'ADJ'), ('English', 'NOUN')),
 (('*-136', 'NOUN'), ('*-136', 'X')),
 (('childish', 'X'), ('childish', 'ADJ')),
 (('fit', 'NOUN'), ('fit', 'ADJ')),
 (('foreigners', 'DET'), ('foreigners', 'NOUN')),
 (('summoned', 'NOUN'), ('summoned', 'VERB')),
 (('about', 'ADP'), ('about', 'ADV')),
 (('8.75', 'VERB'), ('8.75', 'NUM')),
 (('shown', 'X'), ('shown', 'VERB')),
 (('F-series', 'VERB'), ('F-series', 'NOUN')),
 (('Crew', 'X'), ('Crew', 'NOUN')),
 (('Cab', 'VERB'), ('Cab', 'NOUN')),
 (('pickups', 'X'), ('pickups', 'NOUN')),
 (('much', 'ADV'), ('much', 'ADJ')),
 (('mulling', 'X'), ('mulling', 'VERB')),
 (('such', 'ADJ'), ('such', 'DET')),
 (('more', 'ADJ'), ('more', 'ADV')),
 (('damaged', 'VERB'), ('damaged', 'ADJ')),
 (('genes', 'VERB'), ('genes', 'NOUN')),
 (('that', 'ADP'), ('that', 'DET')),
 (('*T*-114', 'DET'), ('*T*-114', 'X')),
 (('Cancer', 'X'), ('Cancer', 'NOUN')),
 (('accrued', 'NOUN'), ('accrued', 'VERB')),
 (("C'mon", 'NOUN'), ("C'mon", 'VERB')),
 (('boyfriends', 'X'), ('boyfriends', 'NOUN')),
 (('review', 'NOUN'), ('review', 'VERB')),
 (('more', 'ADV'), ('more', 'ADJ')),
 (('royalty', 'DET'), ('royalty', 'NOUN')),
 (('Zenith', 'DET'), ('Zenith', 'NOUN')),
 (('534', 'NOUN'), ('534', 'NUM')),
 (('microcomputers', 'DET'), ('microcomputers', 'NOUN')),
 (('84-month', 'NOUN'), ('84-month', 'ADJ')),
 (('2.375', 'NOUN'), ('2.375', 'NUM')),
 (('name', 'NOUN'), ('name', 'VERB')),
 (('hospital', 'DET'), ('hospital', 'NOUN'))]
```

Testing model on modified Viterbi with morphological cues (Best model)

```
start = time.time()
tagged_seq_sm = Viterbi_manual(test_tagged_words)
check = [i for i, j in zip(tagged_seq_sm, test_run_base) if i == j]
accuracy = len(check)/len(tagged_seq_sm)
end = time.time()
print('Time Taken:',end-start)
print('Accuracy is:',accuracy)
print('Laplace smoothed Viterbi-Incorrect tagged cases are:')
incorrect_tagged_cases = [[test_run_base[i-1],j] for i, j in enumerate(zip(tagged_seq_sm,
test_run_base)) if j[0]!=j[1]]
[i[1]for i in incorrect_tagged_cases]
```

```
 (('richer', 'PRON'), ('richer', 'ADJ')),
 (('more', 'ADJ'), ('more', 'ADV')),
 (('galvanized', 'VERB'), ('galvanized', 'ADJ')),
 (('lower-priced', 'VERB'), ('lower-priced', 'ADJ')),
 (('structural', 'NOUN'), ('structural', 'ADJ')),
 (('nearby', 'ADV'), ('nearby', 'ADJ')),
 (('that', 'ADP'), ('that', 'DET')),
 (('cute', 'NOUN'), ('cute', 'ADJ')),
 (('Some', 'DET'), ('Some', 'ADV')),
 (('cyclical', 'NOUN'), ('cyclical', 'ADJ')),
 (('inspirational', 'ADP'), ('inspirational', 'ADJ')),
 (('all', 'DET'), ('all', 'ADV')),
 (('Laura', 'DET'), ('Laura', 'NOUN')),
 (('Dobson', 'ADP'), ('Dobson', 'NOUN')),
 (('freshman', 'DET'), ('freshman', 'NOUN')),
 (('teacher-cadet', 'NOUN'), ('teacher-cadet', 'ADJ')),
 (('Egyptian', 'DET'), ('Egyptian', 'ADJ')),
 (('any', 'DET'), ('any', 'ADV')),
 (('better', 'ADJ'), ('better', 'ADV')),
 (('ago', 'ADP'), ('ago', 'ADV')),
 (('up', 'ADV'), ('up', 'ADP')),
 (('about', 'ADP'), ('about', 'ADV')),
 (("'86", 'NOUN'), ("'86", 'NUM')),
 (('as', 'ADP'), ('as', 'ADV')),
 (('about', 'ADP'), ('about', 'ADV')),
 (('run', 'VERB'), ('run', 'NOUN')),
 (('up', 'ADV'), ('up', 'ADP')),
 (('zero', 'NOUN'), ('zero', 'NUM')),
 (('Montgomery', 'ADP'), ('Montgomery', 'NOUN')),
 (('clear', 'ADJ'), ('clear', 'VERB')),
 (('Arbitrage-related', 'VERB'), ('Arbitrage-related', 'ADJ')
 (('close', 'ADJ'), ('close', 'NOUN')),
 (('plans', 'VERB'), ('plans', 'NOUN')),
 (("C'mon", 'ADP'), ("C'mon", 'VERB')),
 (('refrigerator', 'ADP'), ('refrigerator', 'NOUN')),
 (('review', 'NOUN'), ('review', 'VERB')),
 (('more', 'ADV'), ('more', 'ADJ')),
 (('84-month', 'ADP'), ('84-month', 'ADJ')),
 (('name', 'NOUN'), ('name', 'VERB'))]
```

Testing model on viterbi with unknown word given mostly used tag

```
start = time.time()
tagged_seq_co = Viterbi_common(test_tagged_words)
check = [i for i, j in zip(tagged_seq_co, test_run_base) if i == j]
accuracy = len(check)/len(tagged_seq_co)
```

```
end = time.time()
print('Time Taken:',end-start)
print('Accuracy is :',accuracy)
print('Laplace smoothed Viterbi-Incorrect tagged cases are:')
incorrect_tagged_cases = [[test_run_base[i-1],j] for i, j in enumerate(zip(tagged_seq_co,
test_run_base)) if j[0]!=j[1]]
[i[1]for i in incorrect_tagged_cases]
```

```
Time Taken: 683.8978135585785
Accuracy is : 0.9367861362741237
Laplace smoothed Viterbi-Incorrect tagged cases are:
[(('2.75', 'NOUN'), ('2.75', 'NUM')),
 (('less', 'ADJ'), ('less', 'ADV')),
 (('tunes', 'X'), ('tunes', 'NOUN')),
 (('continental', 'DET'), ('continental', 'ADJ')),
 (('English', 'ADJ'), ('English', 'NOUN')),
 (('*-136', 'NOUN'), ('*-136', 'X')),
 (('childish', 'X'), ('childish', 'ADJ')),
 (('fit', 'NOUN'), ('fit', 'ADJ')),
 (('foreigners', 'DET'), ('foreigners', 'NOUN')),
 (('summoned', 'NOUN'), ('summoned', 'VERB')),
 (('about', 'ADP'), ('about', 'ADV')),
 (('8.75', 'VERB'), ('8.75', 'NUM')),
 (('shown', 'X'), ('shown', 'VERB')),
 (('F-series', 'VERB'), ('F-series', 'NOUN')),
 (('Crew', 'X'), ('Crew', 'NOUN')),
 (('Cab', 'VERB'), ('Cab', 'NOUN')),
 (('pickups', 'X'), ('pickups', 'NOUN')),
 (('much', 'ADV'), ('much', 'ADJ')),
 (('mulling', 'X'), ('mulling', 'VERB')),
 (('such', 'ADJ'), ('such', 'DET')),
 (('more', 'ADJ'), ('more', 'ADV')),
 (('damaged', 'VERB'), ('damaged', 'ADJ'))
```

```
 (('up', 'ADV'), ('up', 'ADP')),
 (('about', 'ADP'), ('about', 'ADV')),
 (("'86", 'X'), ("'86", 'NUM')),
 (('as', 'ADP'), ('as', 'ADV')),
 (('Opus', 'DET'), ('Opus', 'NOUN')),
 (('numbered', 'NOUN'), ('numbered', 'VERB')),
 (('about', 'ADP'), ('about', 'ADV')),
 (('run', 'VERB'), ('run', 'NOUN')),
 (('Mike', 'VERB'), ('Mike', 'NOUN')),
 (('DDB', 'DET'), ('DDB', 'NOUN')),
 (('up', 'ADV'), ('up', 'ADP')),
 (('zero', 'VERB'), ('zero', 'NUM')),
 (('clear', 'ADJ'), ('clear', 'VERB')),
 (('Arbitrage-related', 'NOUN'), ('Arbitrage-related', 'ADJ')),
 (('close', 'ADJ'), ('close', 'NOUN')),
 (('plans', 'VERB'), ('plans', 'NOUN')),
 (('7.8', 'NOUN'), ('7.8', 'NUM')),
 (('Dana-Farber', 'VERB'), ('Dana-Farber', 'NOUN')),
 (('Cancer', 'X'), ('Cancer', 'NOUN')),
 (('accrued', 'NOUN'), ('accrued', 'VERB')),
 (("C'mon", 'NOUN'), ("C'mon", 'VERB')),
 (('boyfriends', 'X'), ('boyfriends', 'NOUN')),
 (('review', 'NOUN'), ('review', 'VERB')),
 (('more', 'ADV'), ('more', 'ADJ')),
 (('royalty', 'DET'), ('royalty', 'NOUN')),
 (('Zenith', 'DET'), ('Zenith', 'NOUN')),
 (('534', 'NOUN'), ('534', 'NUM')),
 (('microcomputers', 'DET'), ('microcomputers', 'NOUN')),
 (('84-month', 'NOUN'), ('84-month', 'ADJ')),
 (('2.375', 'NOUN'), ('2.375', 'NUM')),
 (('name', 'NOUN'), ('name', 'VERB')),
 (('hospital', 'DET'), ('hospital', 'NOUN'))]
```

Testing model on Viterbi choosing only transisition probability in case emission probability is zero

```
start = time.time()
tagged_seq_t = Viterbi(test_tagged_words)
check = [i for i, j in zip(tagged_seq_t, test_run_base) if i == j]
accuracy = len(check)/len(tagged_seq_t)
end = time.time()
print('Time Taken:',end-start)
print('Accuracy is :',accuracy)
print('Laplace smoothed Viterbi-Incorrect tagged cases are:')
incorrect_tagged_cases = [[test_run_base[i-1],j] for i, j in enumerate(zip(tagged_seq_t, test_run_base))
if j[0]!=j[1]]
[i[1]for i in incorrect_tagged_cases]
```

```
Time Taken: 677.2162427902222
Accuracy is : 0.9367861362741237
Laplace smoothed Viterbi-Incorrect tagged cases are:
[(('2.75', 'NOUN'), ('2.75', 'NUM')),
 (('less', 'ADJ'), ('less', 'ADV')),
 (('tunes', 'X'), ('tunes', 'NOUN')),
 (('continental', 'DET'), ('continental', 'ADJ')),
 (('English', 'ADJ'), ('English', 'NOUN')),
 (('*-136', 'NOUN'), ('*-136', 'X')),
 (('childish', 'X'), ('childish', 'ADJ')),
 (('fit', 'NOUN'), ('fit', 'ADJ')),
 (('foreigners', 'DET'), ('foreigners', 'NOUN')),
 (('summoned', 'NOUN'), ('summoned', 'VERB')),
 (('about', 'ADP'), ('about', 'ADV')),
 (('8.75', 'VERB'), ('8.75', 'NUM')),
 (('shown', 'X'), ('shown', 'VERB')),
 (('F-series', 'VERB'), ('F-series', 'NOUN')),
 (('Crew', 'X'), ('Crew', 'NOUN')),
 (('Cab', 'VERB'), ('Cab', 'NOUN')),
 (('pickups', 'X'), ('pickups', 'NOUN')),
 (('much', 'ADV'), ('much', 'ADJ')),
 (('mulling', 'X'), ('mulling', 'VERB')),
 (('such', 'ADJ'), ('such', 'DET')),
 (('more', 'ADJ'), ('more', 'ADV')),
```

```
(("'86", 'X'), ("'86", 'NUM')),
(('as', 'ADP'), ('as', 'ADV')),
(('Opus', 'DET'), ('Opus', 'NOUN')),
(('numbered', 'NOUN'), ('numbered', 'VERB')),
(('about', 'ADP'), ('about', 'ADV')),
(('run', 'VERB'), ('run', 'NOUN')),
(('Mike', 'VERB'), ('Mike', 'NOUN')),
(('DDB', 'DET'), ('DDB', 'NOUN')),
(('up', 'ADV'), ('up', 'ADP')),
(('zero', 'VERB'), ('zero', 'NUM')),
(('clear', 'ADJ'), ('clear', 'VERB')),
(('Arbitrage-related', 'NOUN'), ('Arbitrage-related', 'ADJ'))
(('close', 'ADJ'), ('close', 'NOUN')),
(('plans', 'VERB'), ('plans', 'NOUN')),
(('7.8', 'NOUN'), ('7.8', 'NUM')),
(('Dana-Farber', 'VERB'), ('Dana-Farber', 'NOUN')),
(('Cancer', 'X'), ('Cancer', 'NOUN')),
(('accrued', 'NOUN'), ('accrued', 'VERB')),
(("C'mon", 'NOUN'), ("C'mon", 'VERB')),
(('boyfriends', 'X'), ('boyfriends', 'NOUN')),
(('review', 'NOUN'), ('review', 'VERB')),
(('more', 'ADV'), ('more', 'ADJ')),
(('royalty', 'DET'), ('royalty', 'NOUN')),
(('Zenith', 'DET'), ('Zenith', 'NOUN')),
(('534', 'NOUN'), ('534', 'NUM')),
(('microcomputers', 'DET'), ('microcomputers', 'NOUN')),
(('84-month', 'NOUN'), ('84-month', 'ADJ')),
(('2.375', 'NOUN'), ('2.375', 'NUM')),
(('name', 'NOUN'), ('name', 'VERB')),
(('hospital', 'DET'), ('hospital', 'NOUN'))]
```

Finding Incorrect tagged cases in vanilla Viterbi algorigtm which are corrected in other models

```
incorrect_tagged_cases = [j for i, j in enumerate(zip(tagged_seq_vanilla, test_run_base)) if j[0]!=j[1]]
#inc2=[j[2] for i, j in enumerate(zip(test_run_base, tagged_seq_vanilla,tagged_seq_sm)) if
(j[0]!=j[1])&(j[2][0]==j[1][0])]
pd.set_option('display.max_rows', None)
cor_word=pd.Series([i[1][0] for i in incorrect_tagged_cases])
cor_pos=pd.Series([i[1][1] for i in incorrect_tagged_cases])
inco=pd.Series([i[0][1] for i in incorrect_tagged_cases])
m2=[j[2] for i, j in enumerate(zip(test_run_base, tagged_seq_vanilla, tagged_seq_smooth)) if
(j[0]!=j[1])&(j[2][0]==j[1][0])]
```

```
m2_pos=pd.Series([i[1] for i in m2])
m3=[j[2] for i, j in enumerate(zip(test_run_base, tagged_seq_vanilla,tagged_seq_sm)) if
(j[0]!=j[1])&(j[2][0]==j[1][0])]
m3_pos=pd.Series([i[1] for i in m3])
m4=[j[2] for i, j in enumerate(zip(test_run_base, tagged_seq_vanilla,tagged_seq_co)) if
(j[0]!=j[1])&(j[2][0]==j[1][0])]
m4_pos=pd.Series([i[1] for i in m4])
m5=[j[2] for i, j in enumerate(zip(test_run_base,tagged_seq_vanilla,tagged_seq_t)) if
(j[0]!=j[1])&(j[2][0]==j[1][0])]
m5_pos=pd.Series([i[1] for i in m5])
df=pd.DataFrame (cor_word,columns=['Word'])
df.set_index('Word')
df['Correct_POS']=cor_pos
df['Model1_Viterbi_Vanilla']=inco
df['Model2_Laplace']=m2_pos
df['Mode13 Morphological']=m3_pos
df['Model4_MostTagBased']=m4_pos
df['Mode15_onlyTransistion']=m5_pos
df.drop_duplicates()
```

	Word	Correct_POS	Model1_Viterbi_Vanilla	Model2_Laplace	Mode13 Morphological	Model4_MostTagBased	Mode15_onlyTransistion
0	2.75	NUM	ADV	NOUN	NUM	NOUN	NOUN
1	less	ADV	ADJ	ADJ	ADJ	ADJ	ADJ
2	tunes	NOUN	ADV	X	VERB	X	X
3	carillons	NOUN	ADV	NOUN	ADP	NOUN	NOUN
4	continental	ADJ	ADV	DET	ADP	DET	DET
5	English	NOUN	ADJ	ADJ	ADJ	ADJ	ADJ
6	*-136	X	ADV	NOUN	X	NOUN	NOUN
7	childish	ADJ	ADV	X	NOUN	X	X
8	fit	ADJ	NOUN	NOUN	NOUN	NOUN	NOUN
9	foreigners	NOUN	ADV	DET	ADP	DET	DET
10	summoned	VERB	ADV	NOUN	VERB	NOUN	NOUN
11	about	ADV	ADP	ADP	ADP	ADP	ADP
12	8.75	NUM	ADV	VERB	NUM	VERB	VERB
13	shown	VERB	ADV	X	NOUN	X	X
14	collateral	NOUN	ADV	NOUN	ADP	NOUN	NOUN
15	installing	VERB	ADV	VERB	VERB	VERB	VERB
16	F-series	NOUN	ADV	VERB	VERB	VERB	VERB
17	Crew	NOUN	ADV	X	NOUN	X	X
18	Cab	NOUN	ADV	VERB	NOUN	VERB	VERB
19	pickups	NOUN	ADV	X	NOUN	X	X
20	much	ADJ	ADV	ADV	ADV	ADV	ADV
21	mulling	VERB	ADV	X	VERB	X	X
22	such	DET	ADJ	ADJ	ADJ	ADJ	ADJ
23	mitigate	VERB	ADV	VERB	ADP	VERB	VERB
451	Inns	NOUN	ADV	NOUN	NOUN	NOUN	NOUN
453	zero	NUM	ADV	VERB	NOUN	VERB	VERB
454	Montgomery	NOUN	ADV	NOUN	ADP	NOUN	NOUN
455	clear	VERB	ADJ	ADJ	ADJ	ADJ	ADJ
456	Arbitrage-related	ADJ	ADV	NOUN	VERB	NOUN	NOUN
458	close	NOUN	ADJ	ADJ	ADJ	ADJ	ADJ
460	7.8	NUM	ADV	NOUN	NUM	NOUN	NOUN
461	Dana-Farber	NOUN	ADV	VERB	NOUN	VERB	VERB
462	Cancer	NOUN	ADV	X	NOUN	X	X
463	accrued	VERB	ADV	NOUN	VERB	NOUN	NOUN
464	C'mon	VERB	ADV	NOUN	ADP	NOUN	NOUN
465	boyfriends	NOUN	ADV	X	NOUN	X	X
466	columns	NOUN	ADV	NOUN	NOUN	NOUN	NOUN
467	clipped	VERB	ADV	VERB	VERB	VERB	VERB
468	refrigerator	NOUN	ADV	NOUN	ADP	NOUN	NOUN
471	royalty	NOUN	ADV	DET	NOUN	DET	DET
472	rock	NOUN	ADV	NOUN	NOUN	NOUN	NOUN
473	Zenith	NOUN	ADV	NOUN	NOUN	NOUN	NOUN
474	Zenith	NOUN	ADV	DET	NOUN	DET	DET
475	534	NUM	ADV	NOUN	NUM	NOUN	NOUN
476	microcomputers	NOUN	ADV	DET	NOUN	DET	DET
477	84-month	ADJ	ADV	NOUN	ADP	NOUN	NOUN
478	2.375	NUM	ADV	NOUN	NUM	NOUN	NOUN
479	name	VERB	NOUN	NOUN	NOUN	NOUN	NOUN
480	hospital	NOUN	ADV	DET	NOUN	DET	DET

EXPERIMENT-7
Cocke-Kasami-Younger (CYK)

```
!pip install binarytree

import itertools
from tabulate import tabulate
import math
from collections import defaultdict
import numpy as np
from binarytree import Node
#Class for grammar rule
class Rule:
   def __init__ (self, left, right, prob = 0):
      self.left = left
      self.right = right
      self.prob = prob
# NLP_LAB7.ipynb - Colaboratory
# Class grammer that holds the rule
class Grammar:
   def __init__ (self, rules):
      self.grammar = {}
      self.reversed_grammar = {}
      for rule in rules:
         self.grammar[rule.left] = self.grammar.get(rule.left, set()).union(set([rule.right]))
         #self.reversed_grammar[rule.right] = self.reversed_grammar.get(rule.right,
set()).union(str(rule.left) + "##" + str(rule.prob))
         if rule.right not in self.reversed_grammar:
            self.reversed_grammar[rule.right] = defaultdict(int)
         self.reversed_grammar[rule.right][str(rule.left) + "##" + str(rule.prob)] += 1
# Custom Class for binary tree. It inherites the 'Node' class of binarytree package. This is useful to
visualise a tree data structure
class TreeNode (Node):
   def __init__ (self, value, log_prob, left, right, word=None):
      value = value if word is None else f"{value}-{word}"
      super().__init__ (value, left, right)
      # Calculate the log_score using left, right tree
      left_score = 0 if self.left is None else self.left.score
      right_score = 0 if self.right is None else self.right.score
      self.score = log_prob + left_score + right_score
      # Calculate probability
      self.prob = math.exp(self.score)
   def __repr__ (self):
      return str(round(self.prob, 4))
```

```
# Calculates a marginal probability of trees
def marginalize_trees (trees):
    if not trees:
        return 0
    prob = sum([tree.prob for tree in trees])
    return prob
class CKYParser(Grammar):
    # initialize parser
    def __init__ (self, rules):
        super().__init__ (rules)
    # cky parser algorithm
    def parse(self, words):
        # initialize table

        table = [[""] * i + [set()] * (len(words) - i) for i in range(len(words))]
        #
        for j in range(len(words)): # iterate of the columns
            cell = set()
            for word_tag in self.reversed_grammar.get(words[j], {}):
                w, p = word_tag.split('##')
                cell.add (w)
            # initialize diagonal element
            table[j][j] = cell
        # fill up the rows from bottom to up
            for i in reversed(range(j)):
                cell = set()
                for k in range(i, j):
                    rows = table[i] [k]
                    cols = table[k + 1][j]
                    permutations = list(itertools.product(rows, cols))
                    # iterate on each permutation
                    for left, right in permutations:
                        check = f"{left} {right}"
                        if check in self.reversed_grammar.keys():
                            cell = cell.union([k.split('##') [0] for k in self.reversed_grammar[check]])
                # update cell value
                table[i][j] = cell if len(cell)>0 else 'Φ'
        return table
    # weighted cky parser algorithm
    def weigheted_parse(self, words):
        # initialize table
        table = [[""] * i + [set()] * (len(words) - i) for i in range(len(words))]
        tree_list = list()
        # iterate of the columns
        for j in range(len(words)):
            cell = set()
```

```
            for word_tag in self.reversed_grammar.get(words[j], {}):
                w, p = word_tag.split('##')
                p = float (p)
                node = TreeNode(w, np.log(p), left=None, right=None, word=words [j])
                cell.add((w, node))
            # initialize diagonal element
            table[j][j] = cell
        # fill up the rows from bottom to up
            for i in reversed(range(j)):
                cell = set()
                for k in range(i, j):
                    rows = table[i] [k]
                    cols = table[k + 1][j]
                    permutations = list(itertools.product(rows, cols))
                    # iterate on each permutation
                    for left, right in permutations:
                        l_const, l_node = left
                        r_const, r_node = right
                        check = f"{l_const} {r_const}"
                        if check in self.reversed_grammar.keys():
                            for k_rule in self.reversed_grammar[check]:
                                w, p = k_rule.split('##')
                                p = float(p)
                                new_node = TreeNode(w, np.log(p), left=l_node, right=r_node)
                                cell.add((w, new_node))
                table[i][j] = cell
        # Only add trees that have the root node as the start symbol(S)
        for tag, root in table[0][-1]:
            if tag == "S":
                tree_list.append(root)
        return table, tree_list
# NLP_LAB7.ipynb - Colaboratory
sentence = "British left waffles on Falklands"
tokens = sentence.split(" ")
rules = [Rule("S", "NP VP"), Rule("NP", "JJ NP"), Rule("VP", "VP NP"), Rule("VP", "VP PP"), Rule("PP", "P
NP"), Rule("NP", "British")] # Added closing bracket for rules list
# Missing rule definitions in the original, adding placeholders for demonstrative purposes
rules.extend([
    Rule("JJ", "British"), Rule("VP", "left"), Rule("NP", "waffles"), Rule("P", "on"), Rule("NP", "Falklands"),
    Rule("NP", "left"), Rule("VP", "waffles")
])
cky_parser = CKYParser (rules)
table = cky_parser.parse(tokens)
print("CKY Table:")
print()
print(tabulate(table, headers=tokens, showindex="always"))
```

```
# The output provided in the original notebook snippet is not Python code but rendered table. Ignoring
that for now.
# CKY Table:
# British left waffles on Falklands
# 0{'NP', יבבי'} S' 'NP'} {'S'} Φ {'S'}
# 1 {'VP' 'NP' {'VP', t'S. Φ {'S', 'VP'}
# 2 {'VP', 'NP'} Φ {'VP'}
# {'P'} {'PP'}
# 4 {'NP'}
sentence = "astronomers saw stars with ears"
tokens = sentence.split(" ")
# The original code has an incomplete rule definition at the end `Rule(`. Closing it with a placeholder
rule.
rules = [Rule("S", "NP VP", 1.0), Rule("PP", "P NP", 1.0), Rule("VP", "V NP", 0.7), Rule("VP", "VP PP",
0.3), Rule("P", "with", 1.0)]
rules.extend([
    Rule("NP", "astronomers", 0.5), Rule("NP", "stars", 0.5), Rule("NP", "ears", 0.5),
    Rule("V", "saw", 1.0)
])
cky_parser = CKYParser(rules)
table, parse_trees = cky_parser.weigheted_parse (tokens)
print("Weighted CKY Table:")
print()
print(tabulate(table, headers=tokens, showindex="always"))
print()
# best tree
best_tree = max(parse_trees, key=lambda node: node.score)
print("Most Probable Parse Tree:")
print()
print('Score:', best_tree.prob)
print(best_tree)
print()
# Marginalize over all trees
print(f"Probability of Sentence marginalized over the trees: {marginalize_trees(parse_trees)}")
```

```
Collecting binarytree
  Downloading binarytree-6.5.1-py3-none-any.whl.metadata (9.8 kB)
Requirement already satisfied: graphviz in /usr/local/lib/python3.12/dist-packages (from binarytree) (0.21)
Requirement already satisfied: setuptools>=60.8.2 in /usr/local/lib/python3.12/dist-packages (from binarytree) (75.2.0)
Collecting setuptools-scm>=5.0.1 (from setuptools-scm[toml]>=5.0.1->binarytree)
  Downloading setuptools_scm-10.0.5-py3-none-any.whl.metadata (6.5 kB)
Collecting vcs-versioning>=1.0.0.dev0 (from setuptools-scm>=5.0.1->setuptools-scm[toml]>=5.0.1->binarytree)
  Downloading vcs_versioning-1.1.1-py3-none-any.whl.metadata (1.6 kB)
Requirement already satisfied: packaging>=20 in /usr/local/lib/python3.12/dist-packages (from setuptools-scm>=5.0.1->setuptools-scm[toml]>=5.0.1->binarytree) (26.0)
Downloading binarytree-6.5.1-py3-none-any.whl (18 kB)
Downloading setuptools_scm-10.0.5-py3-none-any.whl (21 kB)
Downloading vcs_versioning-1.1.1-py3-none-any.whl (79 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 79.9/79.9 kB 831.3 kB/s eta 0:00:00
Installing collected packages: vcs-versioning, setuptools-scm, binarytree
Successfully installed binarytree-6.5.1 setuptools-scm-10.0.5 vcs-versioning-1.1.1
CKY Table:

    British      left         waffles      on    Falklands
--  ------------ ------------ ------------ ----- -----------
 0  {'NP', 'JJ'} {'NP', 'S'}  {'S'}        Φ     {'S'}
 1               {'NP', 'VP'} {'VP', 'S'}  Φ     {'VP', 'S'}
 2                            {'NP', 'VP'} Φ     {'VP'}
 3                                         {'P'} {'PP'}
 4                                               {'NP'}
Weighted CKY Table:

    astronomers    saw          stars          with         ears
--  ------------ ------------ -------------- ------------ ----------------
 0  {('NP', 0.5)} set()        {('S', 0.175)} set()        {('S', 0.0262)}
 1                {('V', 1.0)} {('VP', 0.35)} set()        {('VP', 0.0525)}
 2                             {('NP', 0.5)}  set()        set()
 3                                            {('P', 1.0)} {('PP', 0.5)}
 4                                                         {('NP', 0.5)}

Most Probable Parse Tree:

Score: 0.026249999999999992

        ______S_______________
       /                      \
NP-astronomers           _________VP______
                        /                 \
                     __VP___             __PP___
                    /       \           /       \
                 V-saw    NP-stars    P-with    NP-ears


Probability of Sentence marginalized over the trees: 0.026249999999999992
```

EXPERIMENT-8
Minimum Edit Distance Method

```
def min_edit_distance(str1, str2):
    len_str1 = len(str1) + 1
    len_str2 = len(str2) + 1
    # Create a matrix to store minimum edit distances
    matrix = [[0 for _ in range(len_str2)] for _ in range(len_str1)]
    # Initialize the matrix with distances for the first row and column
    for i in range(len_str1):
        matrix[i][0] = i
    for j in range(len_str2):
        matrix[0][j] = j
    # Fill in the matrix with minimum edit distances
    for i in range(1, len_str1):
        for j in range(1, len_str2):
            cost = 0 if str1[i - 1] == str2[j - 1] else 1
            matrix[i][j] = min(
                matrix[i - 1][j] + 1,  # Deletion
                matrix[i][j - 1] + 1,  # Insertion
                matrix[i - 1][j - 1] + cost  # Substitution
            )
    return matrix[len_str1 - 1][len_str2 - 1]

def correct_spelling(word, candidate_words):
    # Find the word in the candidate list with the minimum edit distance
    min_distance = float('inf')
    corrected_word = ""
    for candidate in candidate_words:
        distance = min_edit_distance(word, candidate)
        if distance < min_distance:
            min_distance = distance
            corrected_word = candidate
    return corrected_word, min_distance

# Example usage:
input_word = "happee" # Replace this with the word you want to correct
candidate_words = ["happy", "happier", "happiness", "happening", "hippie"]
corrected_word, distance = correct_spelling(input_word, candidate_words)
print(f"Original word: {input_word}")
print(f"Corrected spelling: {corrected_word}")
print(f"Minimum edit distance: {distance}")
```

```
Original word: happee
Corrected spelling: happy
Minimum edit distance: 2
```